

Lecture 22: Regression and Gradient Descent

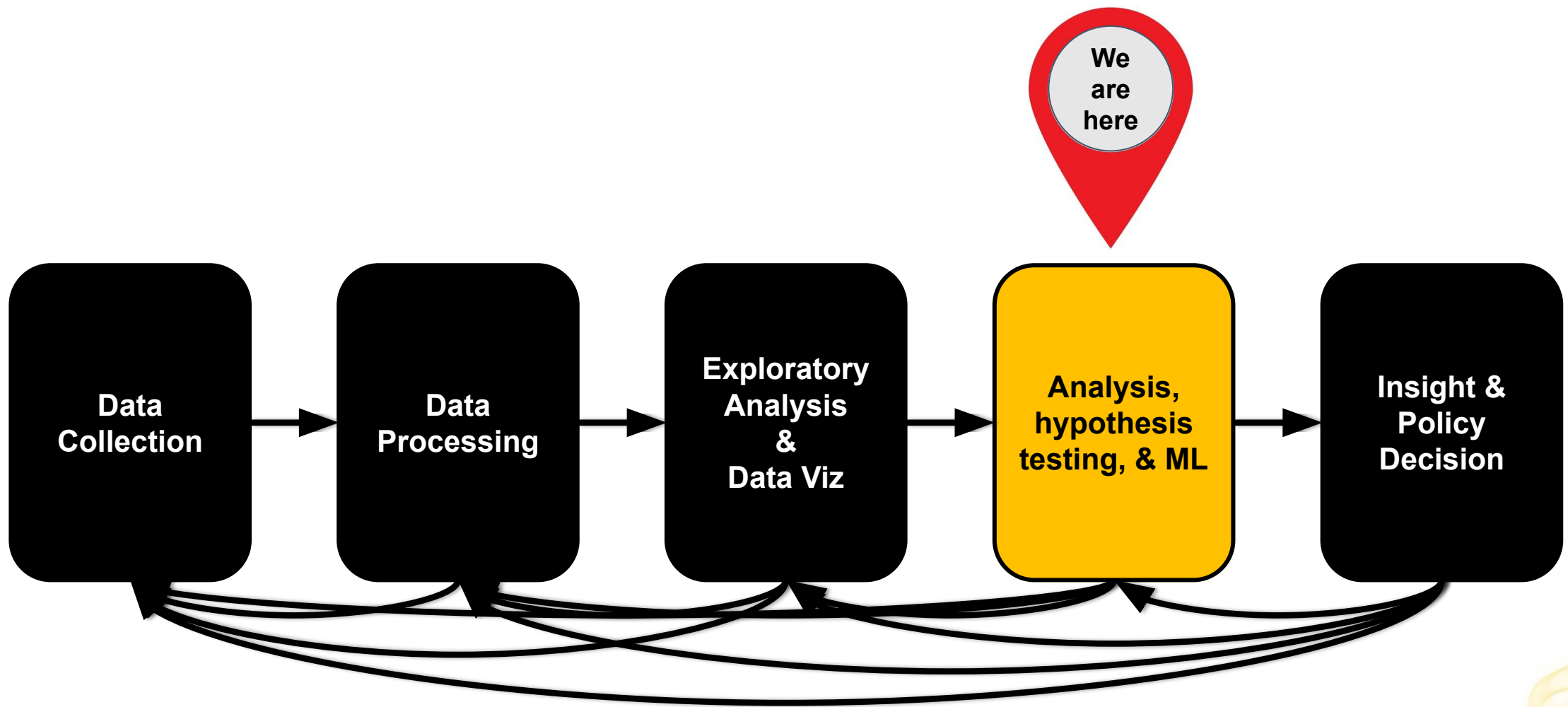
CMSC/DATA 320: Introduction to Data Science

Lecturer: Anna Evtushenko (annae@umd.edu)

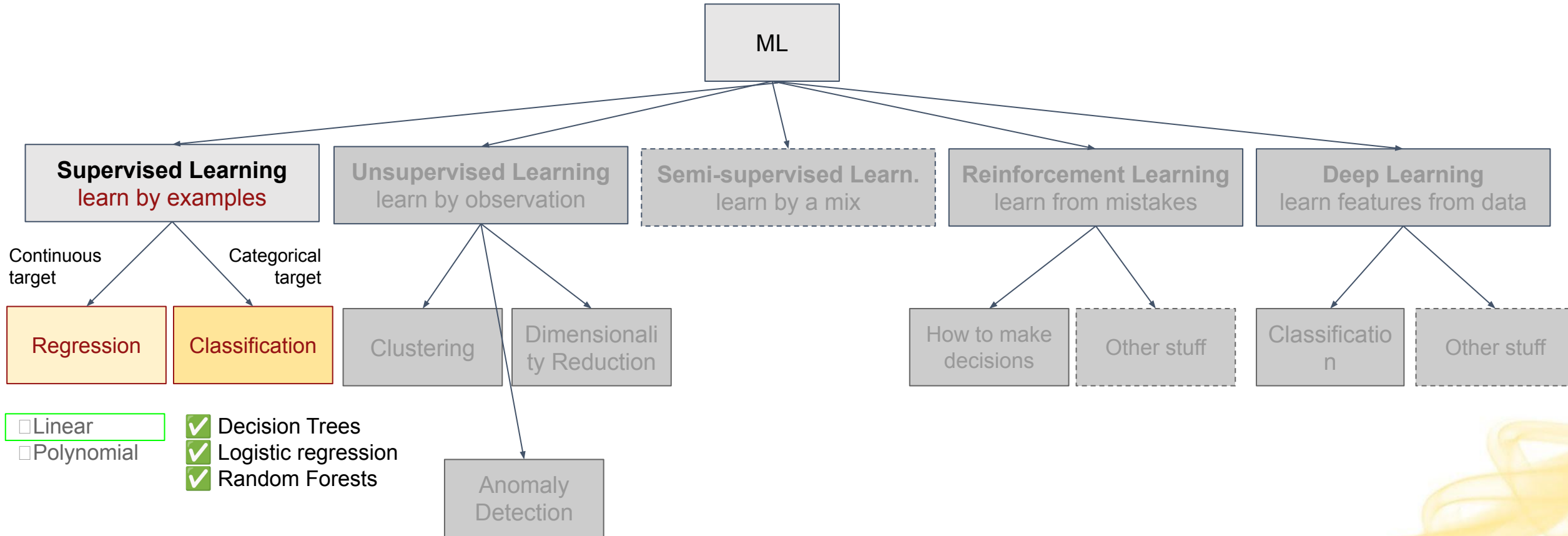
Announcements

- PA6, WA6 due Sun April 26
 - Best to know next week's material (Mon and perhaps Wed)
- Q12 (on Canvas) due tonight at 11:59 pm

Regression



Main categories of Machine Learning



Modular Approach to ML Algorithm Design

- **Choose a model** describing the relationships between variables of interest
- **Define a loss function** quantifying how bad the fit to the data is.
- **Fit the model** that minimizes the loss function and satisfies the constraint/penalty imposed by the regularizer, possibly using an optimization algorithm.

Mixing and matching these modular components gives us many new ML methods.



Remember: No Free Lunch Theorem (NFL)

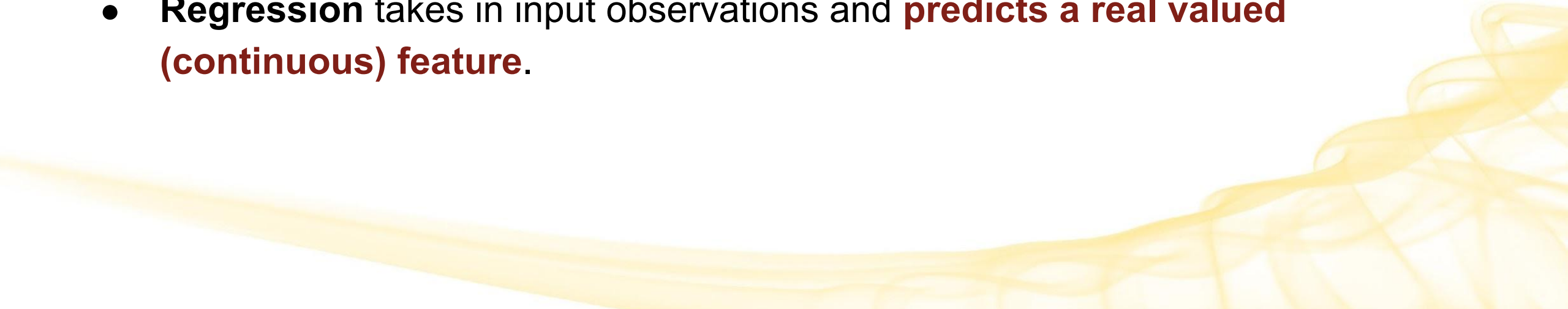
There is no single algorithm that will work well for all tasks.



Regression vs. Classification

What we've been doing so far has been **classification** – predicting the category data will fall into.

The other branch of supervised learning is **regression**.

- **Classification** predicts which **discrete, mutually exclusive category** an observation belongs to.
 - **Regression** takes in input observations and **predicts a real valued (continuous) feature**.
- 

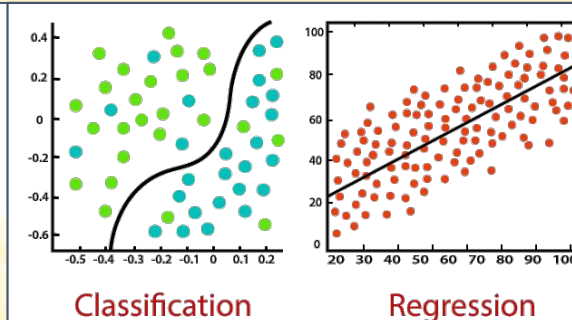
From Classification to Regression

With some **Classification** models, we assumed **linear separability**.

- That a **straight line** (or hyperplane for n -dimensional space) can **divide two classes** in order to make predictions.

With **Regression**, instead of assuming linear separability between classes, we assume a **linear (or other) relationship between features and the output variable**.

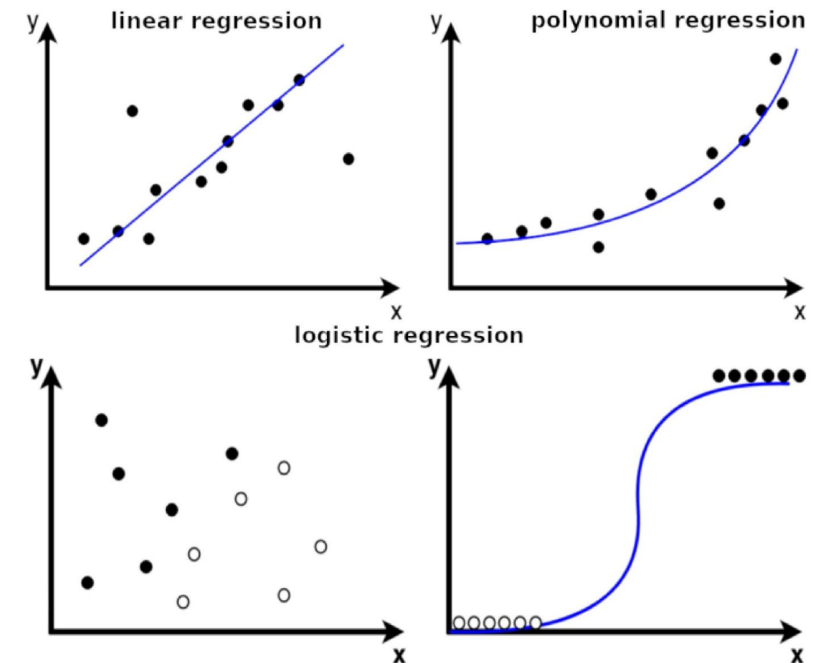
- [**Linear**] That a **straight line** (or hyperplane for n -dimensional space) can **model the relationship** and be used to make predictions
- [**Polynomial**] That a **curve** (with higher order terms) can **model the relationship** and be used to make predictions



Reminder: Regression vs Logistic Regression

Regression is a way to estimate the behavior of an output variable in response to changes in a different input variable, using statistical techniques.

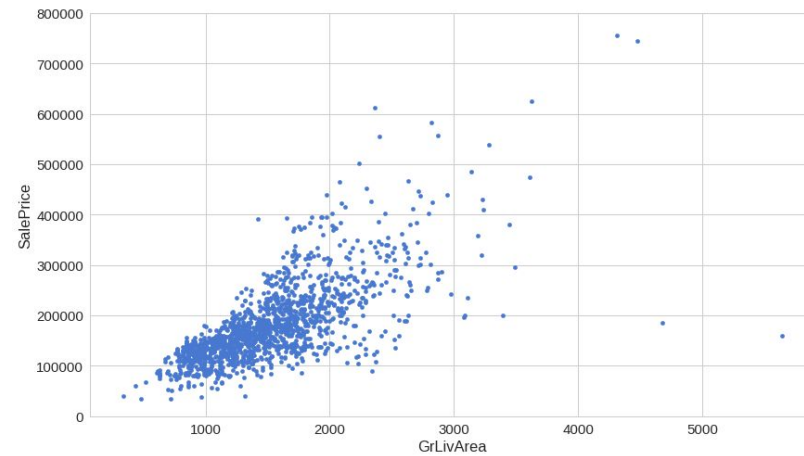
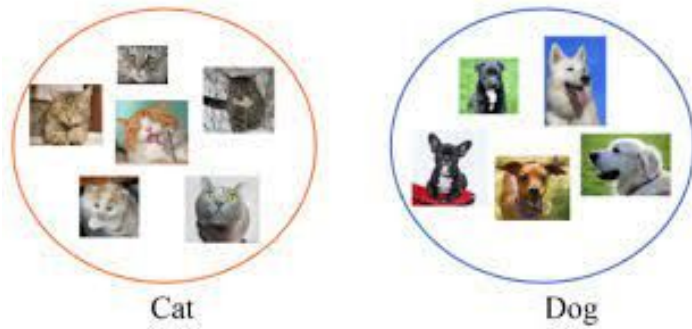
- **Logistic Regression:** algorithm for a **discrete (categorical) output variable**
 - Typically **Binary Classification**
 - Others
- **Regression:** algorithm for a **continuous, normally distributed output variable**
 - **Linear:** tries to fit the relationship to a line
 - **Polynomial:** tries to fit the relationship to a polynomial function
 - Others...



How does Regression differ from Classification?

If you are trying to **decide if something is a cat or a dog**, you **can't be half right**; it's either a cat or a dog.

If you are trying to **predict how much a house will sell for**, you **can be more or less correct**.



Identify: Classification or Regression?



What will be the temperature tomorrow?

84°



Fahrenheit



Will it be hot or cold tomorrow?

COLD

HOT

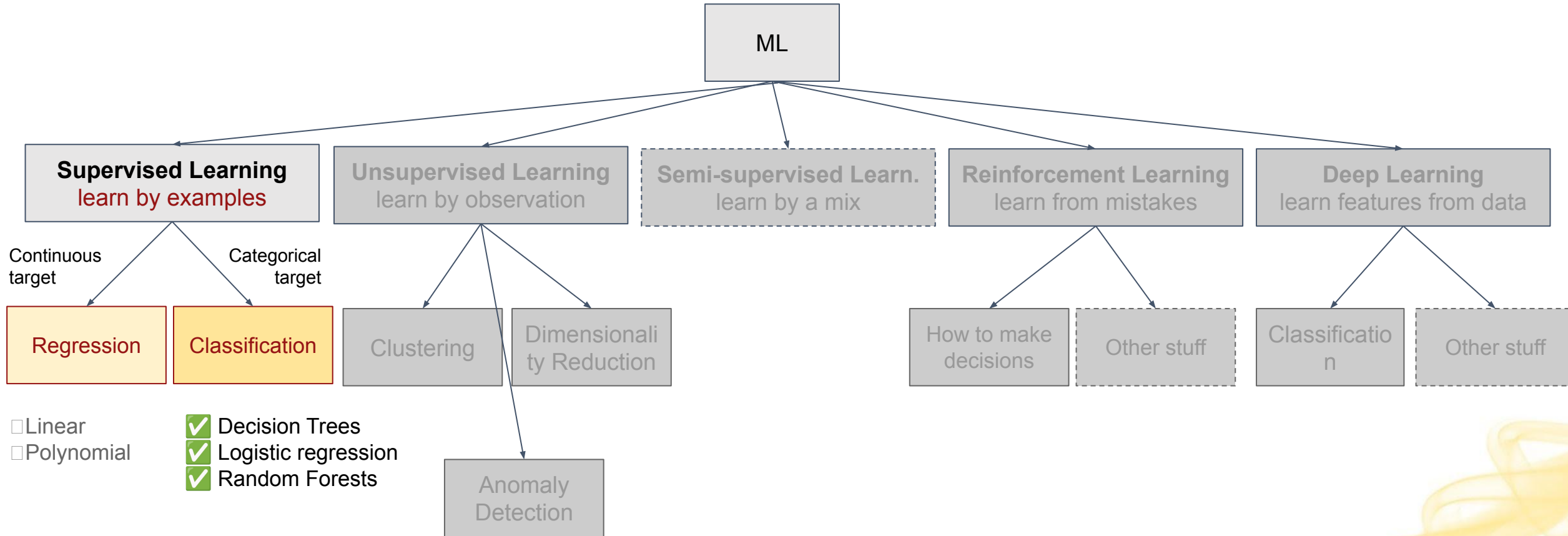


Fahrenheit

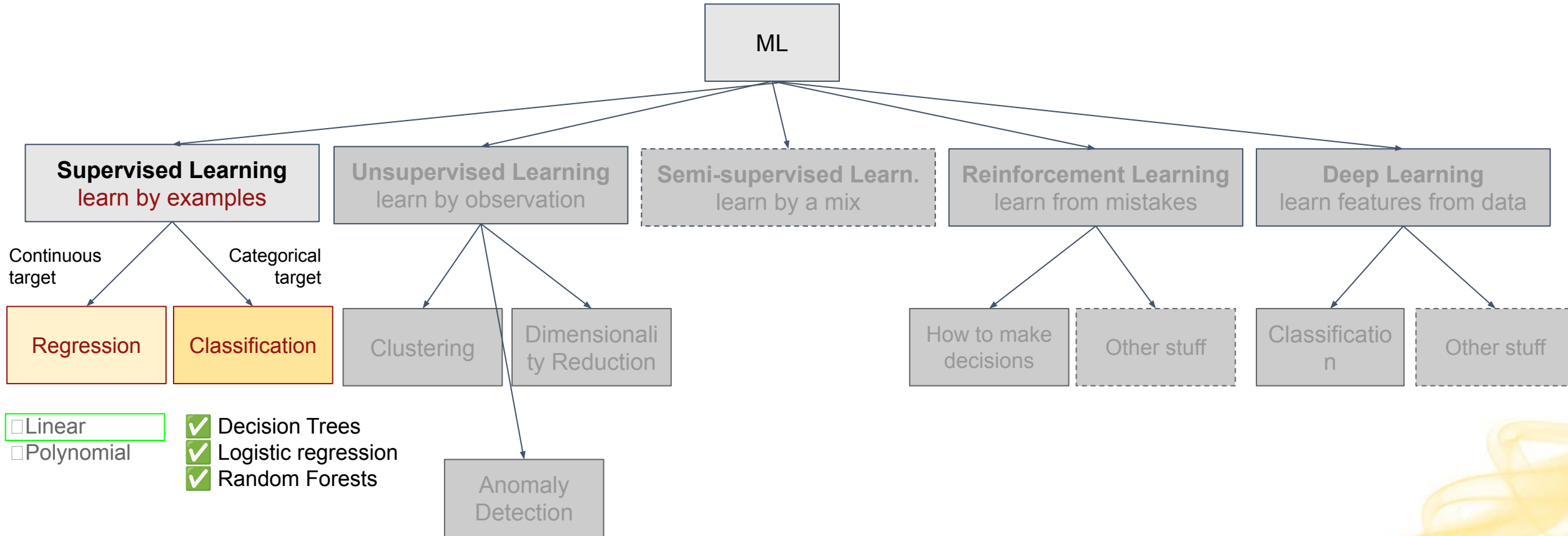
Linear Regression



Main categories of Machine Learning



Main categories of Machine Learning



Problem Setup

Suppose we've collected **Weight and Height measurements from 5 people**, and we want to use **Weight to predict Height**, which is **continuous**.

Person	Weight	Height
1	...	...
2	...	...
3	...	...
4	...	...
5	...	...
6	...	?

We want to know **height** (output) of a new person (observation) based on their **weight** (input)

We can start by using a **simple regression model**.

Problem Setup

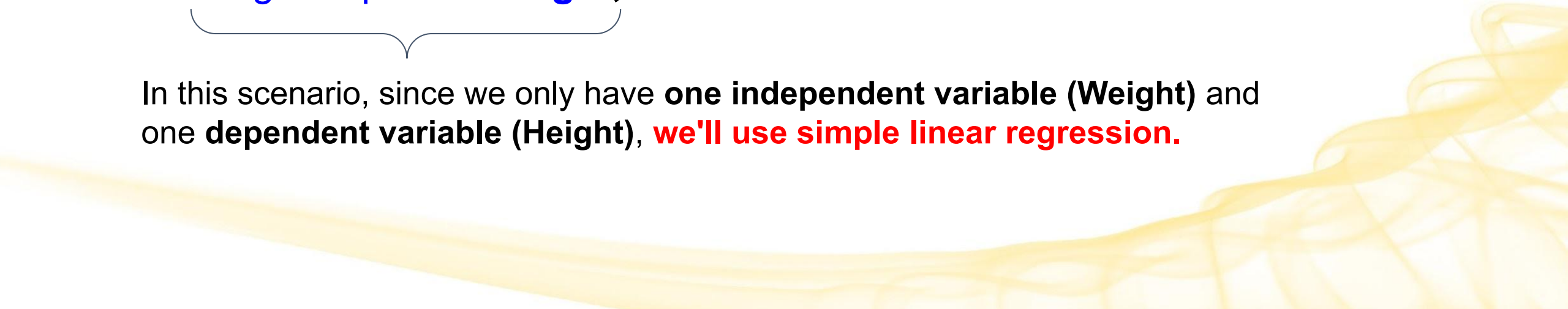
Regression is a statistical method used to **model the relationship between a dependent variable and one or more independent variables.**

Goal: Create a model that can **predict the dependent variable** based on the independent variable(s).

We've collected **Weight and Height measurements from 5 people**, and we want to use **Weight to predict Height**, which is **continuous value**.



In this scenario, since we only have **one independent variable (Weight)** and one **dependent variable (Height)**, **we'll use simple linear regression.**

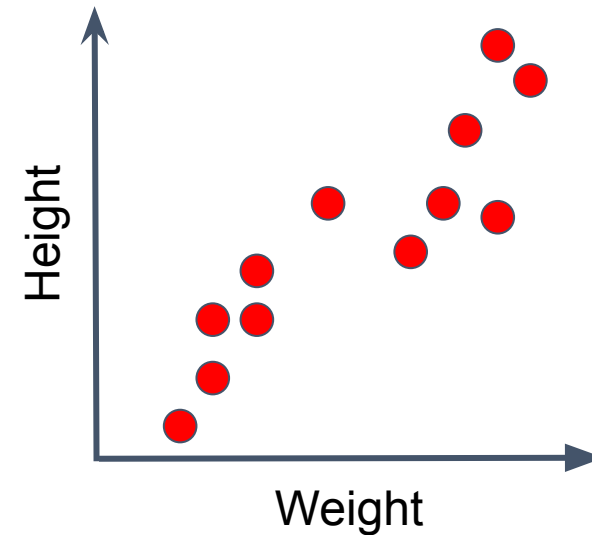
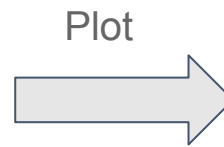


Problem Setup

Want to use **Weight** to **predict Height**, which is continuous.

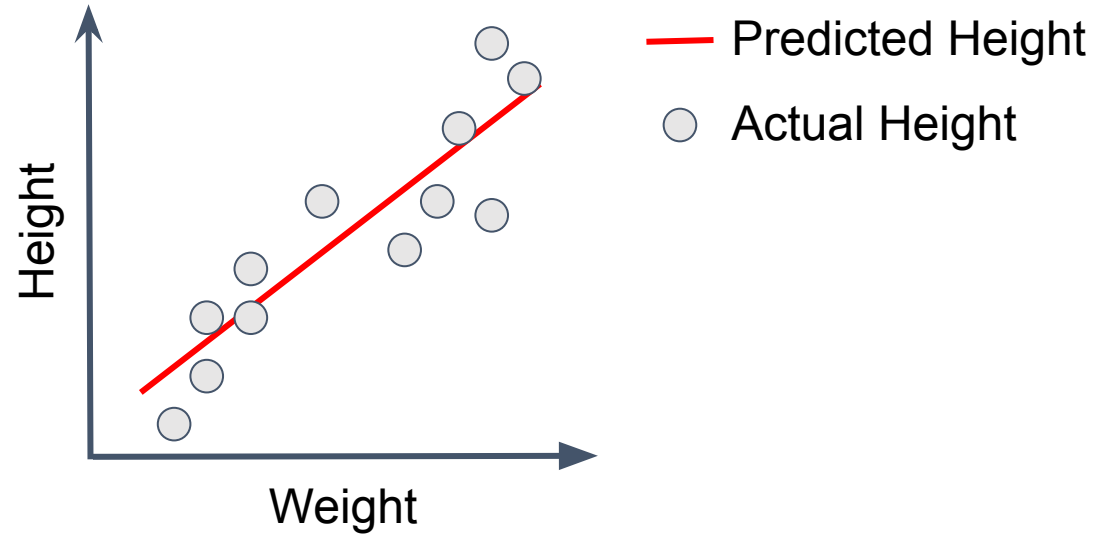
Person	Weight	Height
1	...	...
2	...	...
3	...	...
4	...	...
5	...	...

Training Data



We can plot our data: dependent vs independent variable

In Lay Terms: We want to fit a line

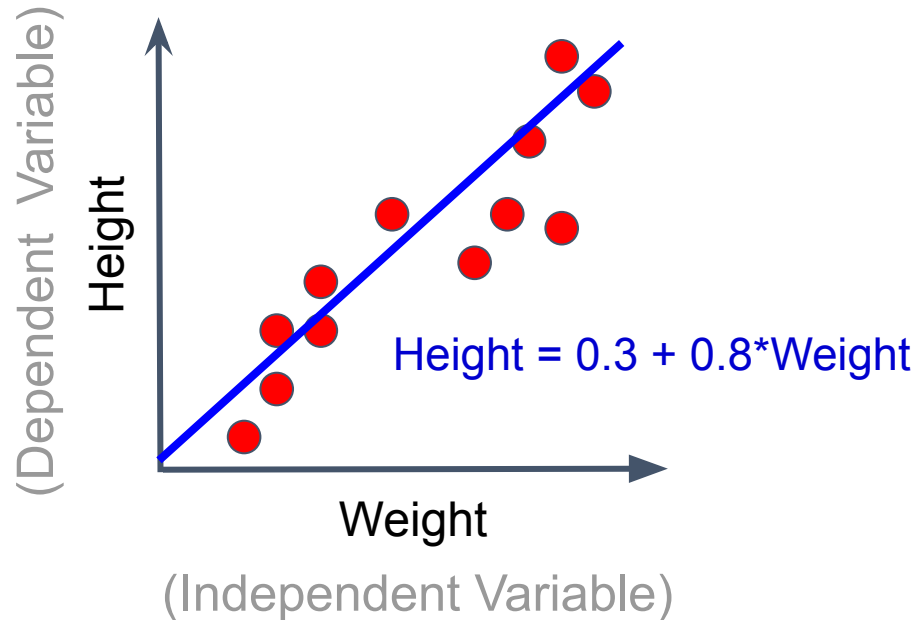


We can ask ourselves:

“What is the line that most nearly passes through all of the data points?”

Linear Equation with 1 Feature

We can **add a line** to the data plot so we can see what the trend is



We want to use **Weight** to **predict Height** (continuous).

Any linear relationship between two variables can be represented as a straight line:

$$y = \square_o + \square_i x$$

Model: y is a linear function of x :

$$y = \square_o + \square_i x$$

Where:

- y = target or prediction $\rightarrow$ Height
- x = feature $\rightarrow$ Weight
- $\square_o$ and $\square_i$ = Model parameters or coefficients
 - $\square_o$ = intercept
 - $\square_i$ = slope = rate of change

What if there are **multiple features**?

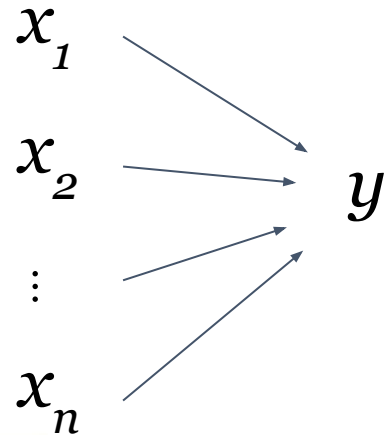


Multiple Features

Linear Regression

$$x \rightarrow y$$

Multiple Linear Regression



In this case we have **multiple independent variables** used to predict a **single dependent variable**.

Linear Equation with Multiple Features

Any linear equation with multiple features can be written as a hyperplane equation:

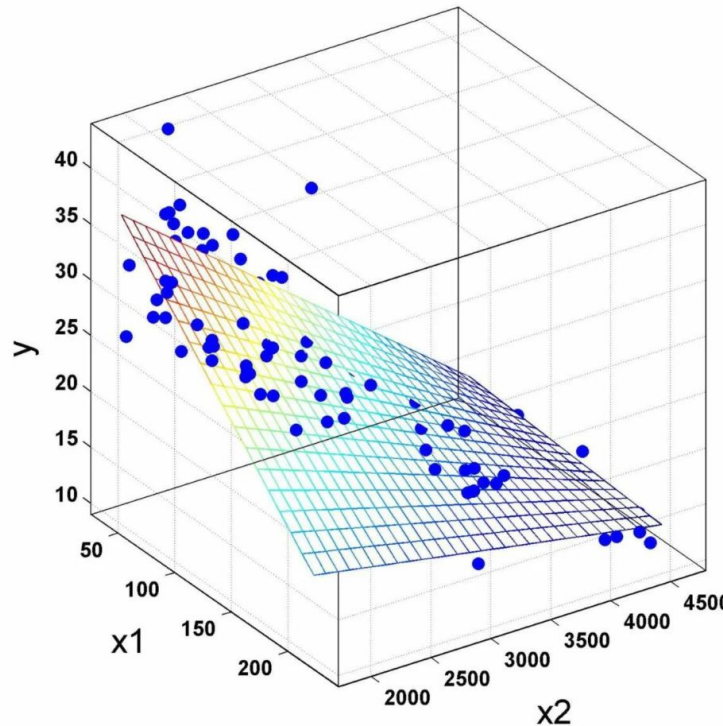
$$y = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \dots \beta_n \cdot x_n$$

- x_i are features (vectors)
- β_i are model parameters (a.k.a coefficients)
- y is the target variable we are predicting

Multiple Linear Regression 3D Example

In 3 dimensions, we have:

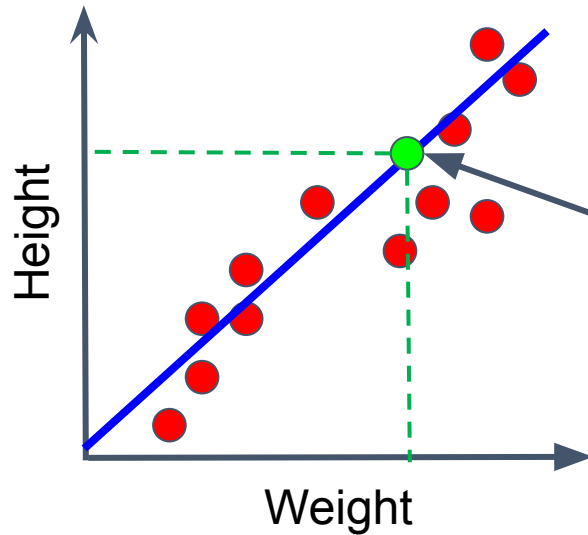
- 1 target variable y
- 2 input features x_1 and x_2



Multiple Linear Regression seeks to find the plane in x_1 vs x_2 vs y that models our predictions.

Back to: Goal of Regression

Find a function that best represents the relationship between the **input variables** (independent variables) and the **output variable** (dependent variable), which is **continuous**.



For a new data point x_i , we want to predict height, based on the weight

Height = $0.3 + 0.8 * \text{Weight}$
Height = $0.3 + 0.8 * 5.1$
Height = 4.38

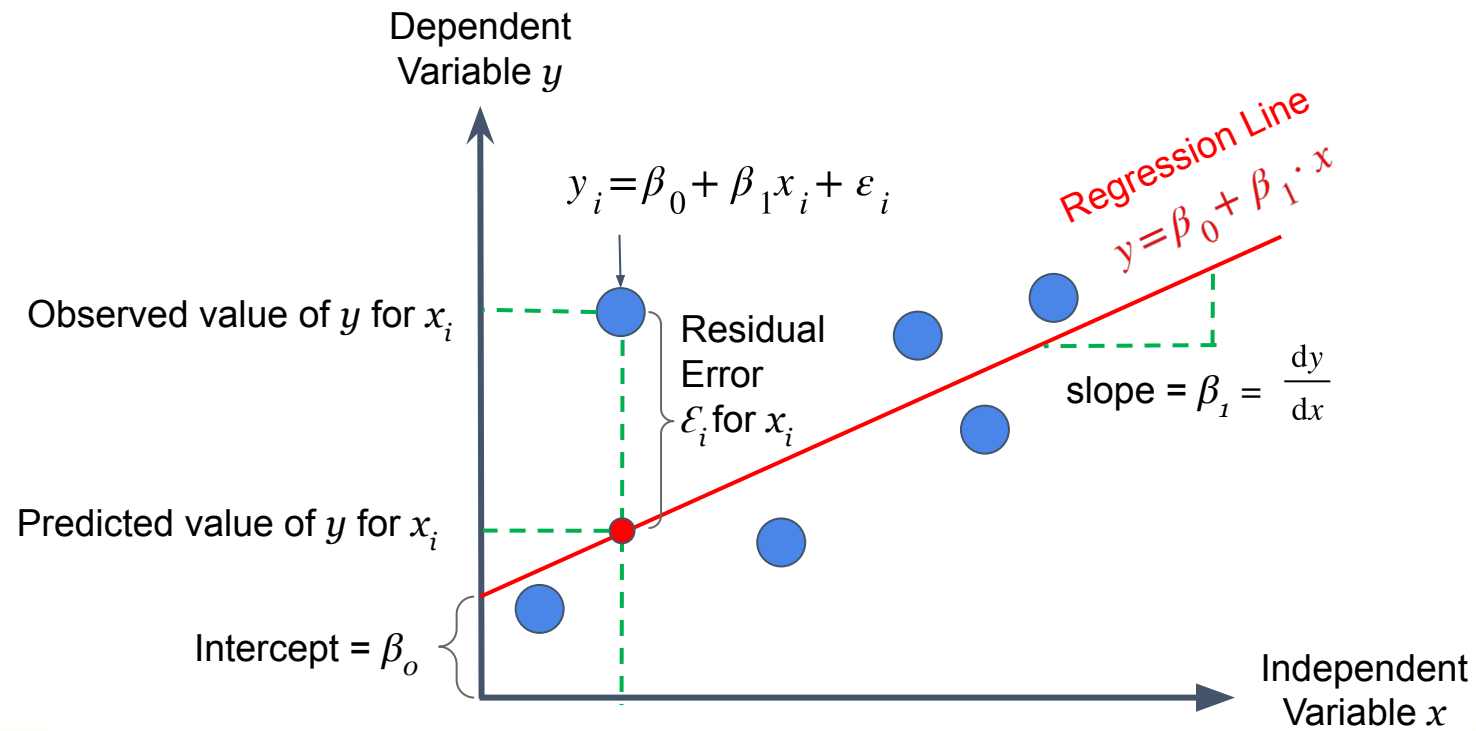
$$y = \square_0 + \square_1 \cdot x$$

β_0 represents the y-intercept of the line
 β_1 represents the slope of the line

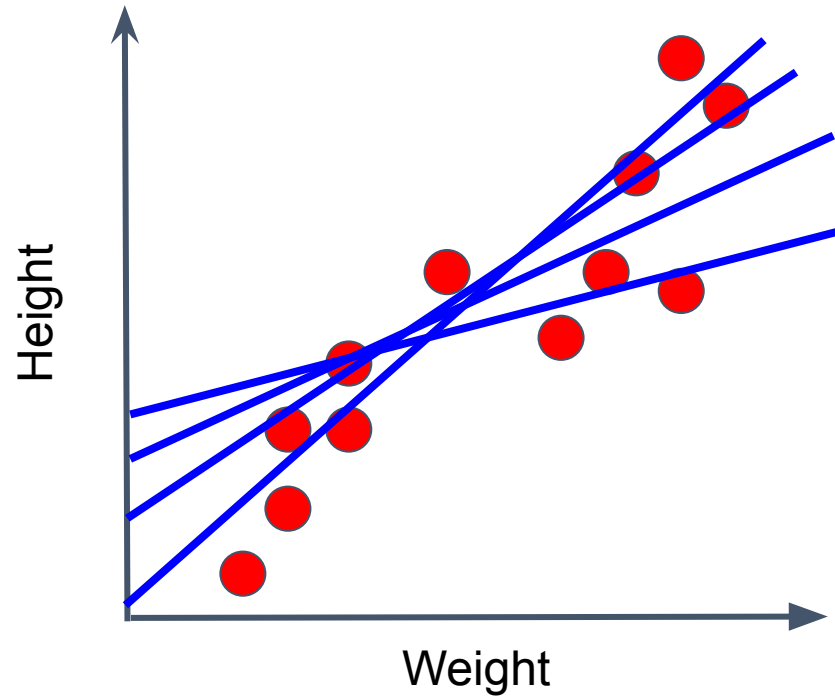
6	...	?
---	-----	---

Regression Line, or Line of Best Fit

The best fit line, often referred to as the **Line of Best Fit** or **Regression Line**, is a straight line that **best represents** the relationship between two variables in a set of data.



But how do we know **which one is the best line?**



How to **find the optimal line** to fit our data?

A reasonable criterion for a line to be the “best” is for it to have the **smallest possible overall error** among all straight lines.

How to know which is the best line, cont.

Why **that particular line**?

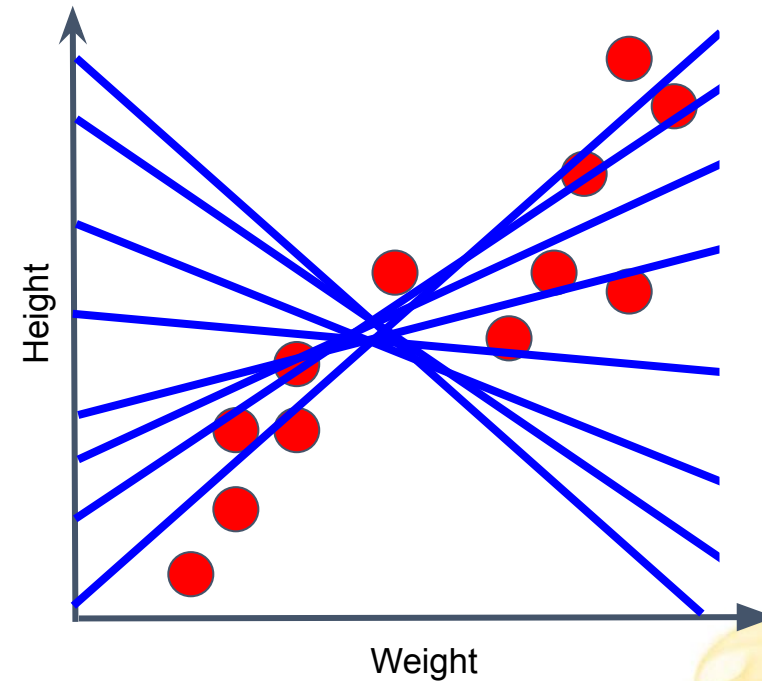
We aim to choose a line that:

- **Best Fits:** The chosen regression line accurately fits the data.
- **Minimizes Errors:** Aims to reduce differences between predicted and actual values.
- **Captures Relationship:** Shows the relationship between variables for accurate predictions.

We need a process of determining the best-fitting line → **Linear Regression Method**

Remember:

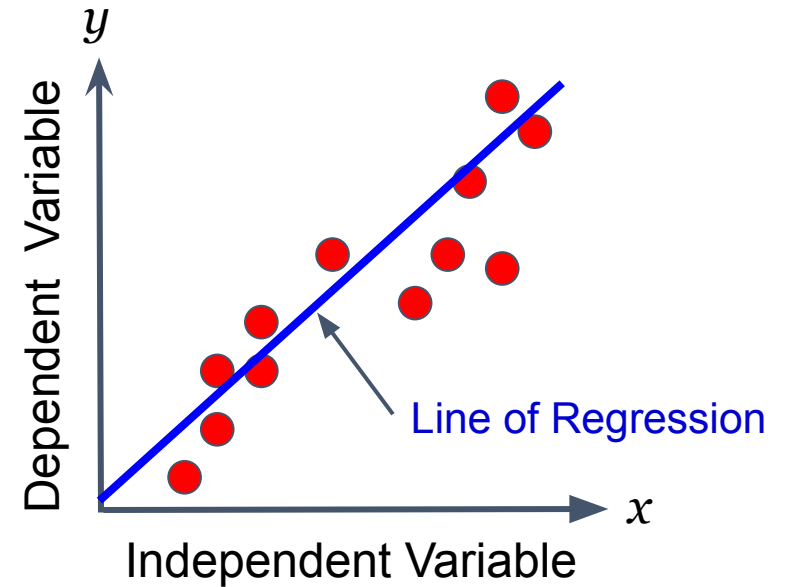
*The best fit line will have the least **training** error.*



Linear Regression

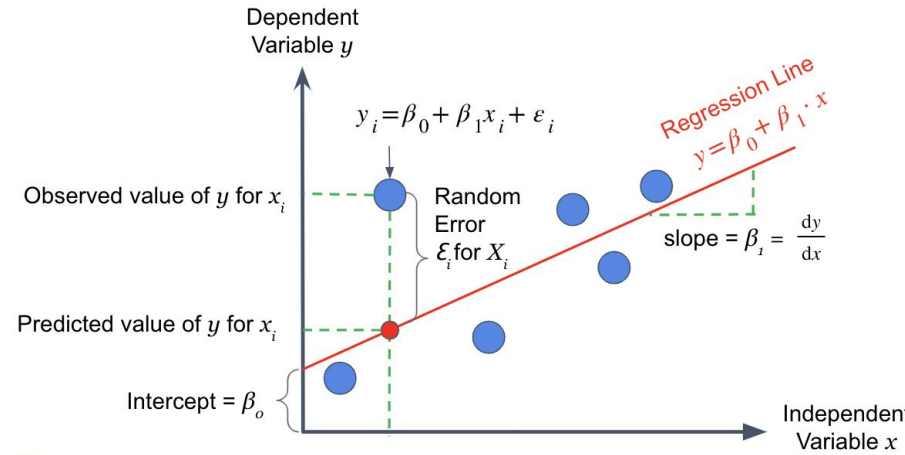
Linear Regression is a statistical method used to **model the relationship between variables** by **finding the best fit line** in order to enable predictions.

- Measures the relationship between **one or more independent variables** vs **one dependent variable**.
 - **Independent variables** → predictors (inputs)
 - **Dependent variables** → predictions (outputs)
- Examples:
 - Years of Experience vs Salary
 - Area vs House Price



Goal of the Linear Regression algorithm

Find the best values for β_0 and β_1 in order to find the best fit line.



The best fit line is a line that has **the least error**, meaning the error between predicted values and actual values **(for the training set)** should be minimal.

Find the best values of β_0 and β_1 to find the best fit line.

To do this, we use the method of **Least Squares** to minimize a **cost function**, typically the **Mean Squared Error (MSE)**, to fit a line to the data.



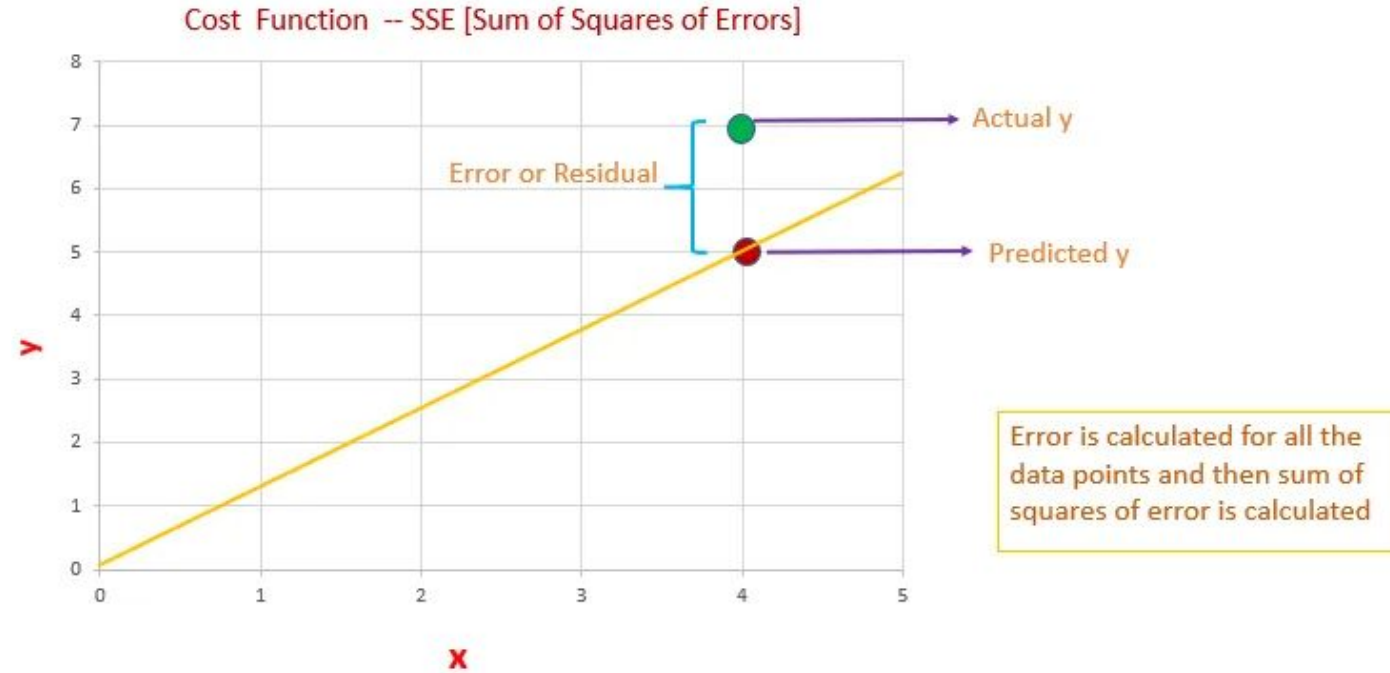
Residual or Error

Error/Residuals: the difference between the *actual value* of y and the *predicted value* of y.

Residuals are signed **vertical distances** from data points to the modeled line (not the shortest distance).

Residual Error:

$$\varepsilon = y_{\text{actual}} - y_{\text{predicted}}$$



A well-fitting model should have **small residuals on average**.

Residual or Error

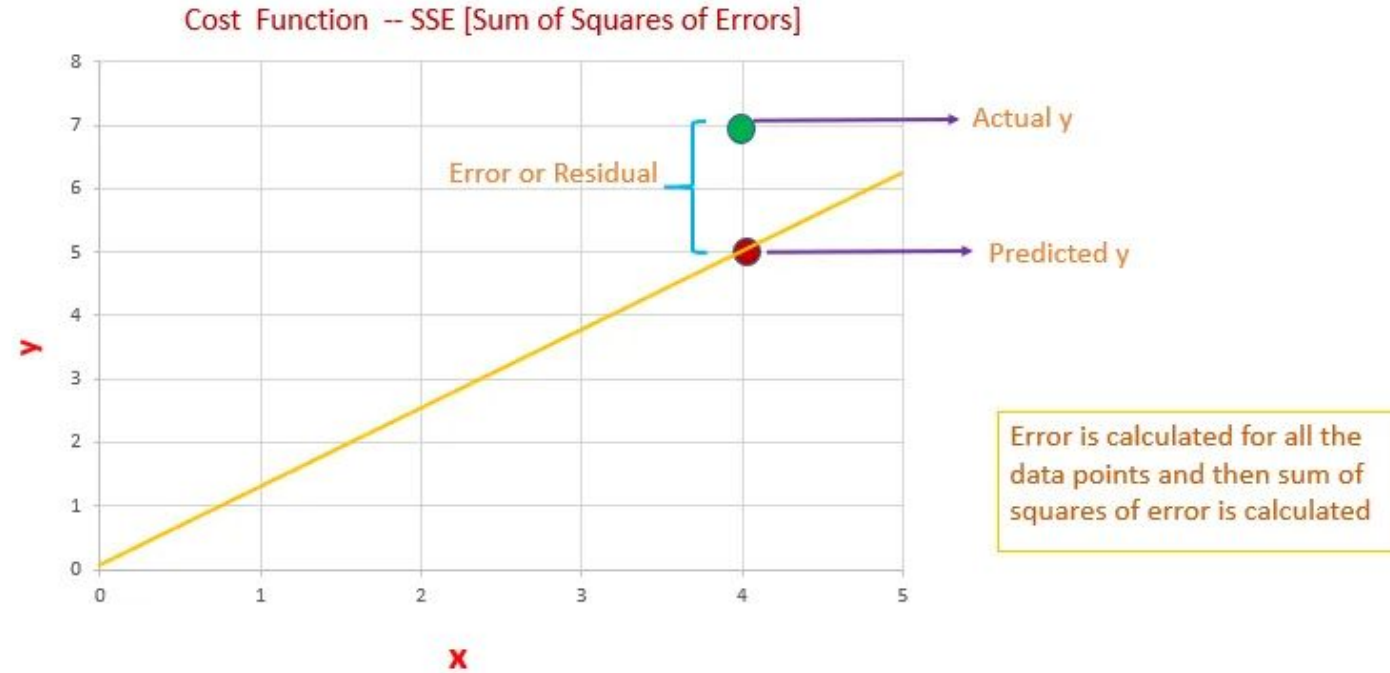
Error/Residuals: the difference between the *actual value* of y and the *predicted value* of y.

Residuals are signed **vertical distances** from data points to the modeled line (not the shortest distance).

Residual Error:

$$\varepsilon = y_{\text{actual}} - y_{\text{predicted}}$$

Usually denoted $\hat{y}$

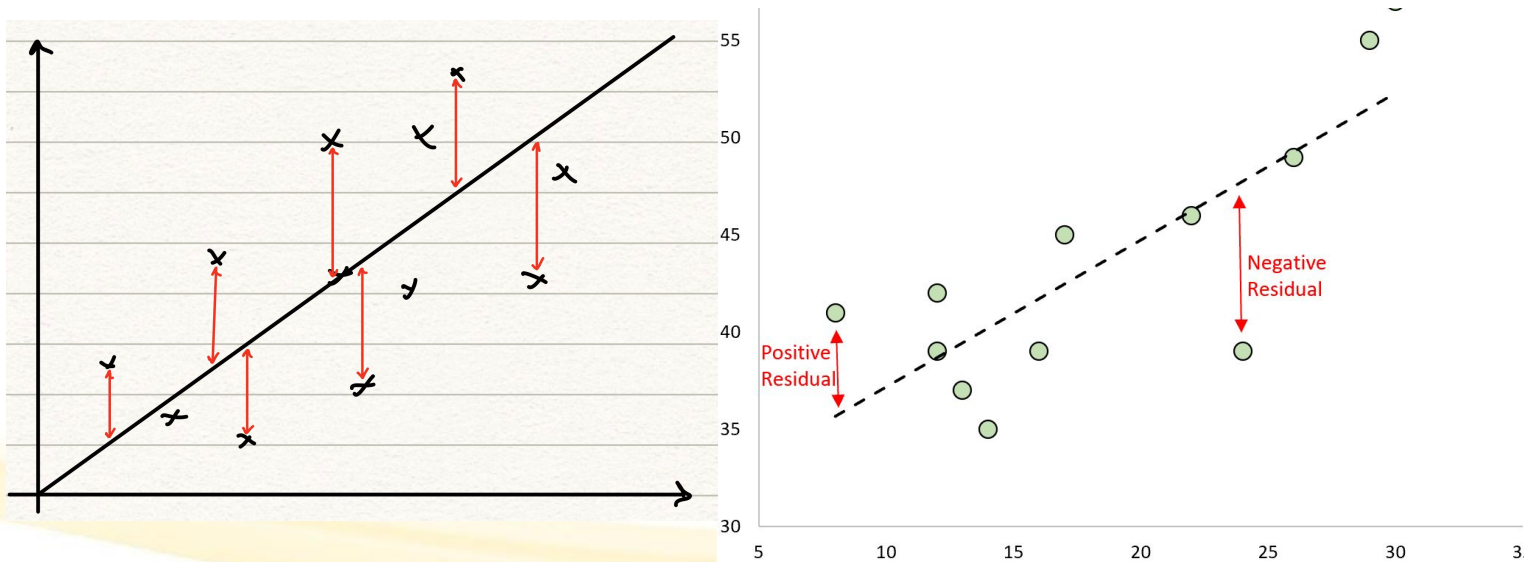


A well-fitting model should have **small residuals on average**.

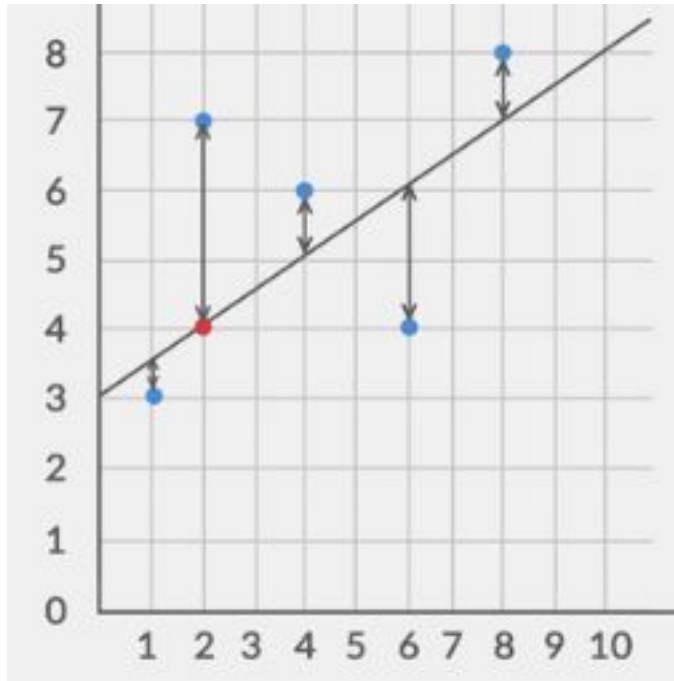
Residual Sum of Squares (RSS) error

Residual Sum of Squares (RSS) is a measure of how good or bad the fit is. Also called the **Sum of Squared Residuals (SSR)** or **Sum of Squared Estimate of Errors (SSE)**.

We can define a **loss function** using the **squared distances (residuals) between the observed data points and the predicted values**, then **sum those squares** to evaluate how well the model fits the data.



Residual Sum of Squares (RSS) error



$$\hat{y}_i = \beta_0 + \beta_1 x_i$$

$$\varepsilon_i = y_i - \hat{y}_i$$

$$\varepsilon_i = y_i - \beta_0 - \beta_1 x_i$$

$$\varepsilon_i^2 = (y_i - \beta_0 - \beta_1 x_i)^2$$

$$RSS = \varepsilon_1^2 + \varepsilon_2^2 + \dots + \varepsilon_n^2 = \sum_{i=1}^n \varepsilon_i^2$$

$$RSS = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

Cost Function: Mean Squared Error (MSE)

Since there are n training examples, we turn this into a Cost Function simply by dividing by n such that this represents an average across all n observations.

This is called the **Mean Squared Error**, or MSE.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

Least-Square (LS) Regression Line

The line

$$\hat{y} = \beta_0 + \beta_1 \cdot x$$

that minimizes

$$J(\beta) = MSE = \frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_0 - \beta_1 x_i \right)^2$$

Least-Square (LS) Regression Line

The line

$$\hat{y} = \beta_0 + \beta_1 \cdot x$$

Hypothesis function →

Equivalent to:

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

that minimizes

$$J(\beta) = MSE = \frac{1}{n} \sum_{i=1}^n \left(y_i - \beta_0 - \beta_1 x_i \right)^2$$

Cost function →

Just like:

$$J(\theta)$$

*Note: These are for the case of 1 independent variable x and 1 dependent variable y .

Now we have an ML Optimization Problem!

Goal: Minimize Cost Function $J(\beta)$ (MSE)

- Our cost function needs to be minimized in order to find the **best values of β_0 and β_1** , in order to find the **best fit prediction line**.
 - This is how we **learn** the coefficients!
- How do we optimize/minimize?
 - With an **optimization algorithm**!

Said another way: in the context of linear regression, **optimization** is employed to **find the optimal coefficients** (slopes and intercept) for the linear equation that **best fits the data**.

Optimization Algorithm

In Machine Learning, an Optimization Algorithm is simply an algorithm that **adjusts the model parameters** (weights) in order to **improve the accuracy** of a model and **reduce errors**.

- This is done by **minimizing a cost function** or **maximizing an objective function**
- There are many different optimization algorithms!
 - Ex. **Gradient Descent**, **Stochastic Gradient Descent**, **Particle Swarm Optimization**, **Conjugate Gradient**... and many more.
- The technique we'll use here is a popular optimization algorithm, known as **Gradient Descent**.
 - GD is an iterative optimization algorithm to find the minimum of a function

Gradient Descent



Rate of Change

When we talked about our line of best fit, we defined a slope, β_1 , that represents the **rate of change** for the prediction line (line of regression).

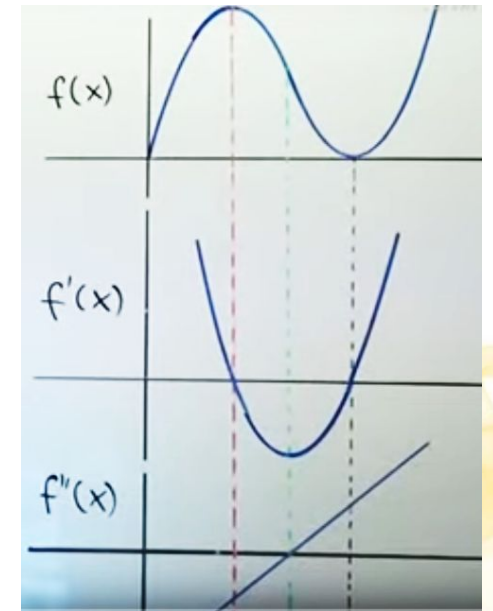
It's given by the rise over the run (slope) of *two points*.

But there is also a notion of rate of change for a *single point*.

- This is the **derivative** of the function at that point.
- This is the **slope** at that given point.

And, we can know the rate of change of the rate of change.

- This is the **second derivative** at that point
- This is how the **point slope changes**

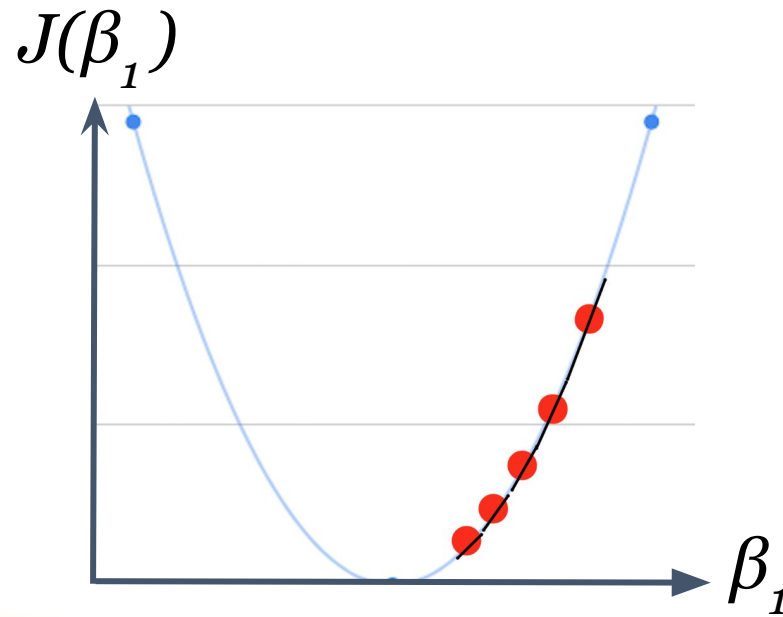


Assumption: the function is differentiable everywhere.

Cost Function

We can use this notion of rate of change, or derivative, at a point to consider how we approach our cost function for a variety of possible coefficient values.

Consider first just β_1 :



We want to **minimize** our cost, which means finding the **minimum** of this $J(\beta)$ function.

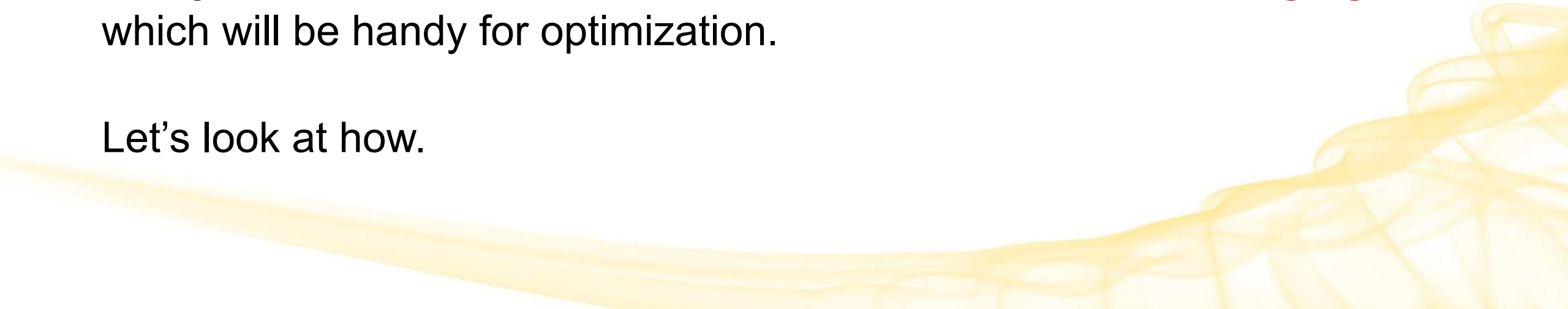
Derivative of our Cost Function

What's nice about this quadratic is that it is a well-behaved function, meaning that it has a well-defined derivative and second derivative.

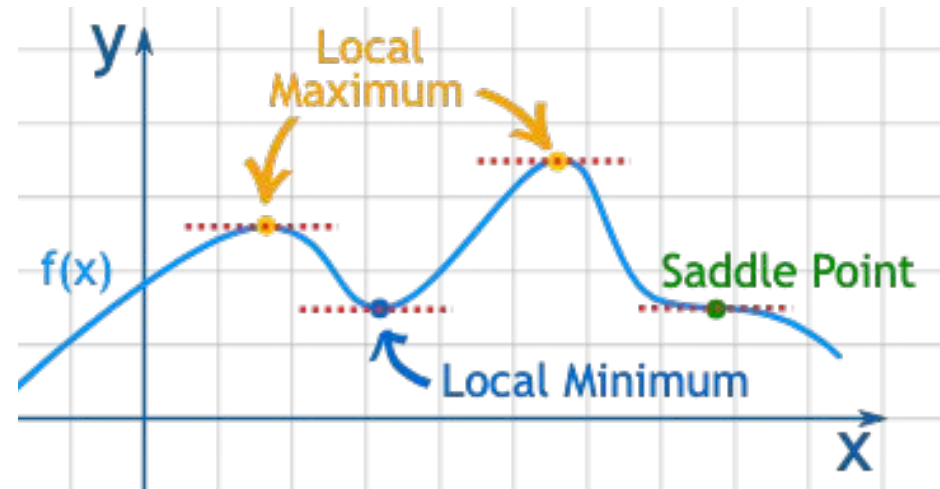
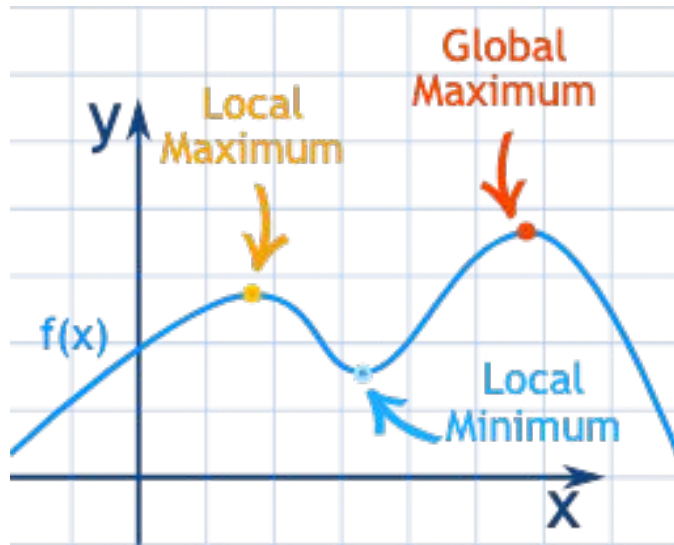
- **Derivative:** a straight line
- **Second derivative:** a constant

Using this, we can understand **how the cost function is changing**, which will be handy for optimization.

Let's look at how.



Maxima and Minima



The derivative is zero at any maximum or minimum

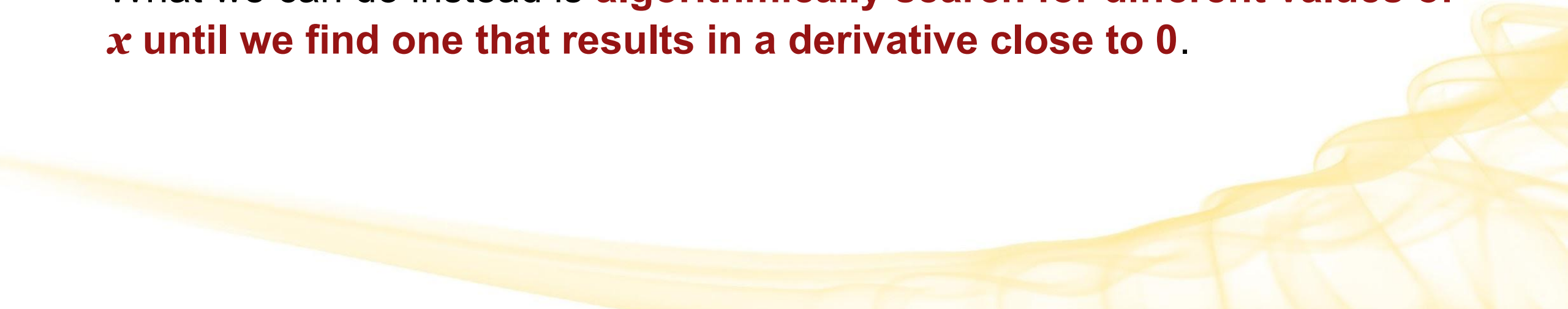


Finding Maxima and Minima

Since the derivative is always zero for maxima or minima, one way we can find them is by setting the function to zero, and then solving for the independent variable, x .

This can be hard in practice.

What we can do instead is **algorithmically search for different values of x until we find one that results in a derivative close to 0.**



But, we have more than one coefficient

- Remember, we have more than just one coefficient.
 - We have two, β_0 and β_1 , even in the simple case of 1 input feature.
 - And for Multiple Linear Regression (more than 1 input feature), we could have *many*.
- Just as we did with Logistic Regression, we'll let θ represent an estimate for our parameters/weights/coefficients, β
 - i.e. let θ be n estimated coefficients $\theta_0 \dots \theta_n$.
- In the general case, our best fit line/hyperplane becomes $\theta_0 + \theta_1 x_1 + \dots \theta_n x_n$
- And just as before, we'll let x_0 be a dummy “feature” that always equals 1 so that we can rewrite as

$$\hat{y} = h_{\theta}(x) = \theta^T x$$

Remember: θ and x are vectors

$$\theta = \begin{bmatrix} \theta_0 \\ \vdots \\ \theta_n \end{bmatrix} \quad x = \begin{bmatrix} x_0 \\ \vdots \\ x_n \end{bmatrix}$$

$$\hat{y} = \theta^T x = \begin{bmatrix} \theta_0 & \dots & \theta_n \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ \vdots \\ x_n \end{bmatrix} = \theta_0 \cdot x_0 + \dots \theta_n \cdot x_n$$

$\hat{y}$ is a scalar value - it is the estimate of the y value at that point.

Cost Function Tweak

It is convenient to multiply our MSE Cost Function by $\frac{1}{2}$ so that when we take the derivative over and over again we don't have to keep multiplying by 2.

This is a common convention to use, and does not impact the minimization functionality.

Thus our cost function becomes:

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{2n} \sum_{i=1}^n (y_i - \theta^T x_i)^2$$

Derivatives of the Cost Function

For the general case, what we want is to find the partial derivatives of our Cost Function with respect to each estimated coefficient, θ_i :

$$\frac{\partial J}{\partial \theta_j} \longrightarrow$$

Tells us the slope of the cost/error curve for that coefficient at a given point.

This is how much the cost is changing and in what direction.

The partial derivative is given by:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot x_{ij} \longrightarrow$$

We can use this to know how to change our estimate of the best θ .

Gradient

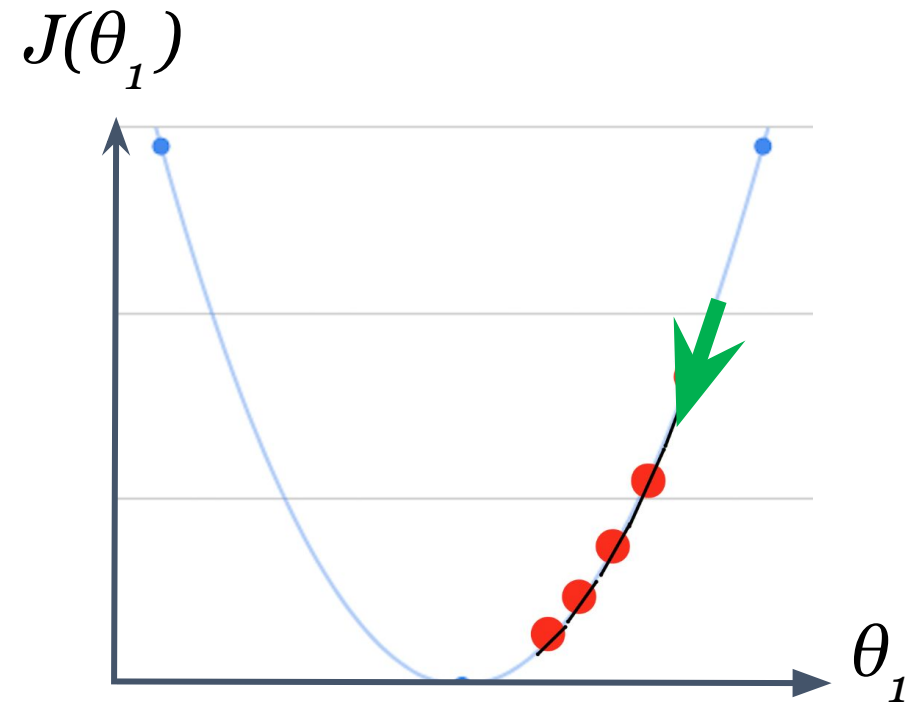
The vector of all partial derivatives of the cost function is known as its **Gradient**, indicated by a ∇ symbol, which is a handy shorthand.

For our cost function, we write this as $\nabla J(\theta)$:

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \vdots \\ \frac{\partial J}{\partial \theta_n} \end{bmatrix} = \frac{1}{n} (y - \theta^T x) \cdot x$$

The gradient is a vector that points in the direction of the steepest **increase** of the cost function at a given point.

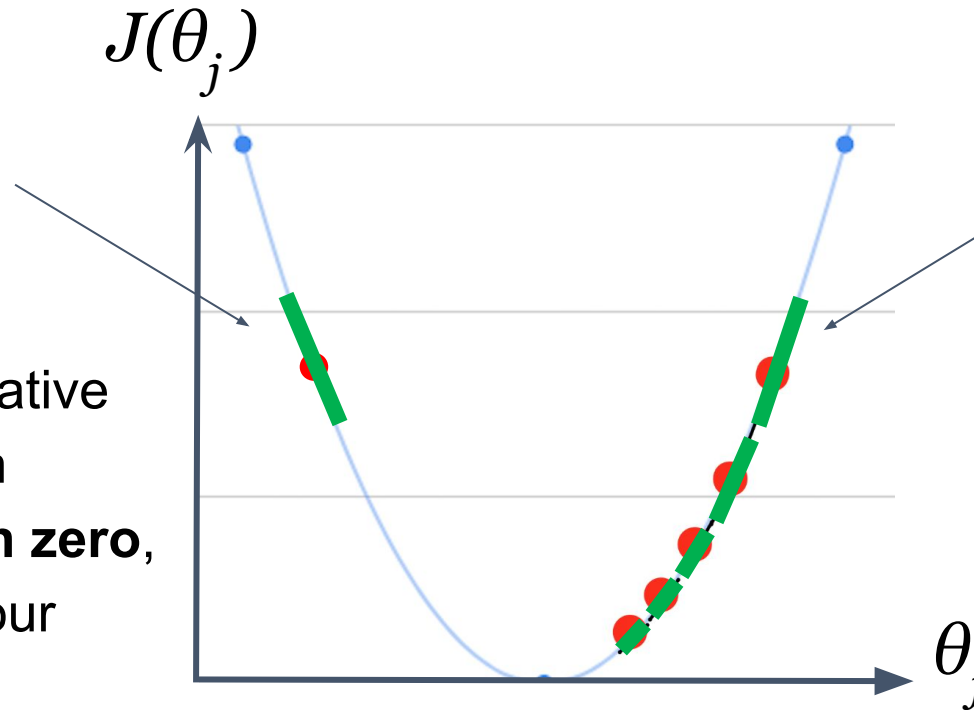
Gradient example



Updating our Estimates

$$\frac{\partial J}{\partial \theta_j} < 0$$

When the partial derivative with respect to a given coefficient is **less than zero**, we want to **increase** our estimate of θ_j



$$\frac{\partial J}{\partial \theta_j} > 0$$

When the partial derivative with respect to a given coefficient is **greater than zero**, we want to **decrease** our estimate of θ_j

Updating our Estimates

Since the gradient represents the direction of steepest **increase**, we can simply update our coefficients in the **opposite direction of the gradient**.

$$\theta_{new} = \theta - \alpha \cdot \nabla J(\theta)$$

This enables us to move towards lower and lower values of the cost function until a minimum is reached.

Updating our Estimates

So, at each iteration, we update all of the weights:

$$\begin{aligned}\theta_0 &= \theta_0 - \alpha \cdot \frac{\partial J(\theta)}{\partial \theta_0} \\ &\vdots \\ \theta_j &= \theta_j - \alpha \cdot \frac{\partial J(\theta)}{\partial \theta_j}\end{aligned}$$

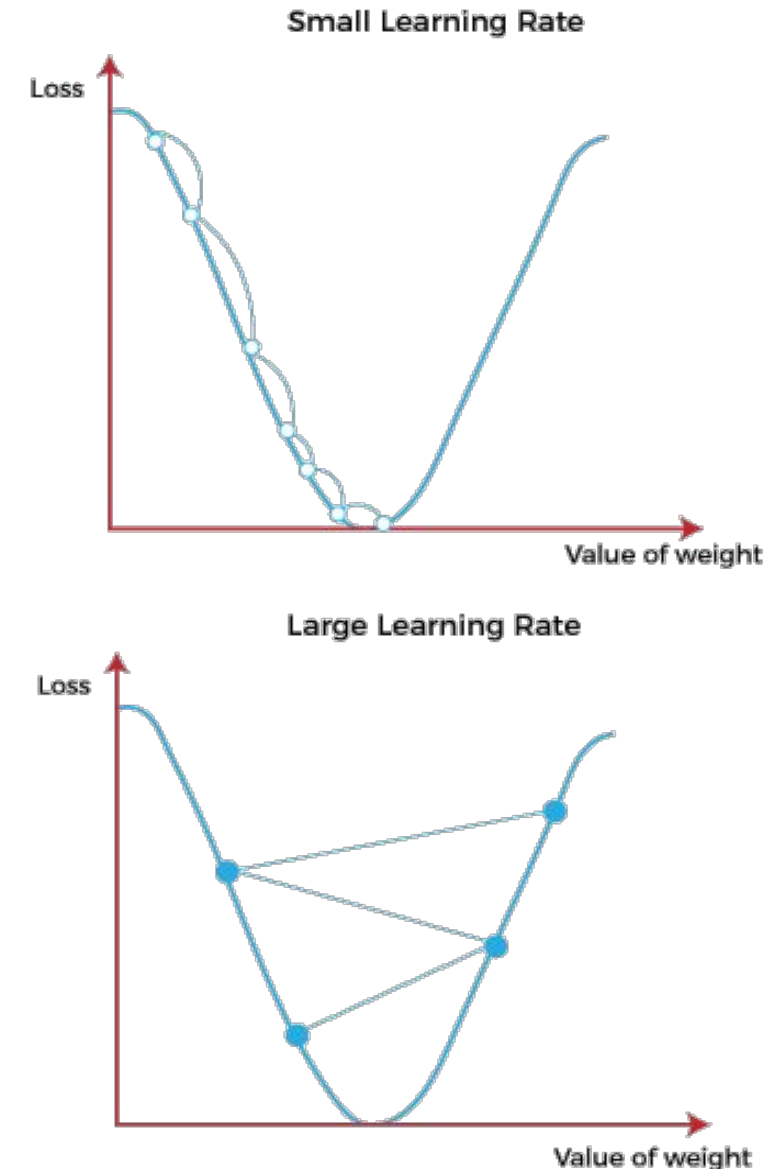


Wait, what is α ??

Learning Rate α

Learning Rate α is a hyperparameter we can tune.

- Determines the size of steps taken in Gradient Descent.
- **Large Learning Rate:** Covers more ground per step, but risks overshooting the minimum.
- **Small Learning Rate:** Ensures precise movement, but may result in slow convergence due to frequent gradient calculations.
- **Common Learning Rates:**
 - 0.001, 0.003, 0.01, 0.03, 0.1, 0.3

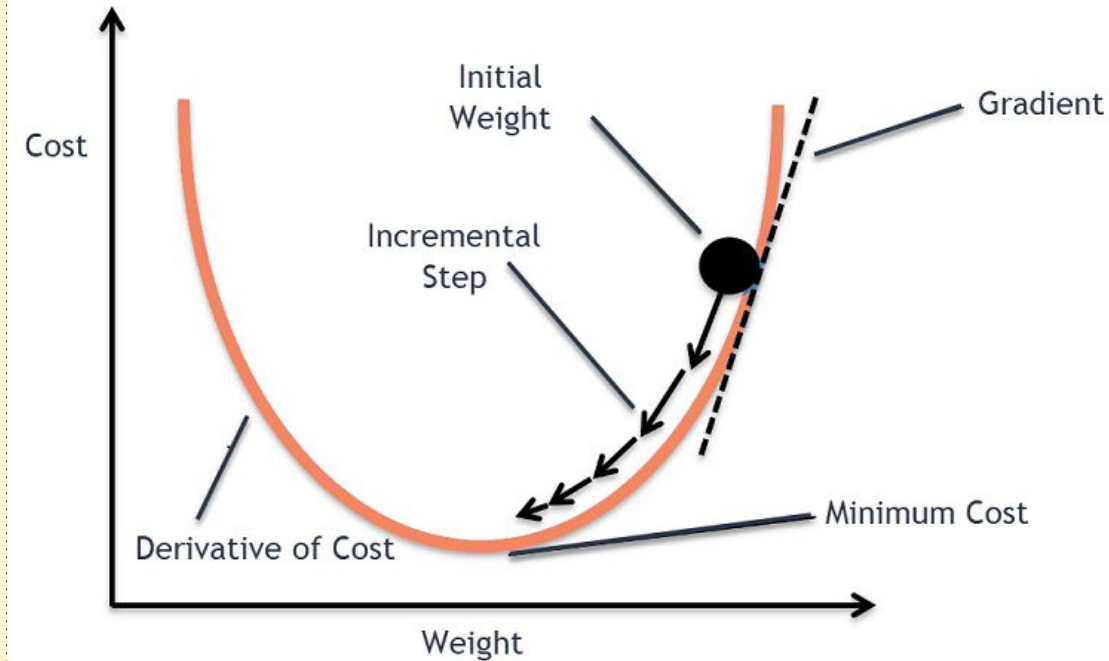


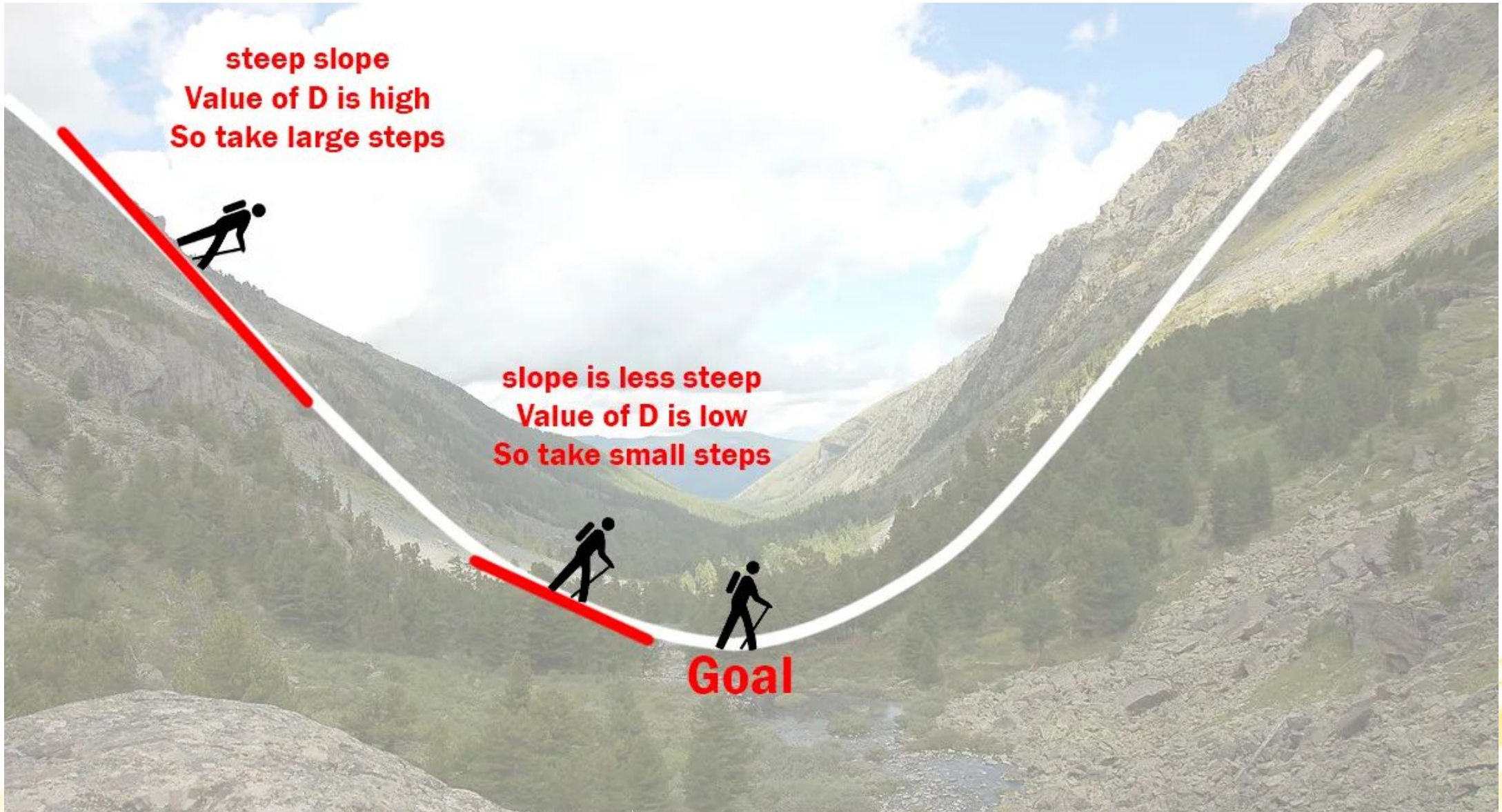
Gradient Descent Algorithm

An **iterative** optimization algorithm that finds optimal parameter values by **minimizing a cost function**.

Process:

- **Initialization:** Start with initial values for θ s.
- **Cost Calculation:** Calculate the cost function.
- **Parameter Adjustment:** Update (increase or decrease) parameter estimates (θ s) on the gradient to find the next lowest cost value.
- **Iteration:** Repeat process until the minimum cost is achieved.

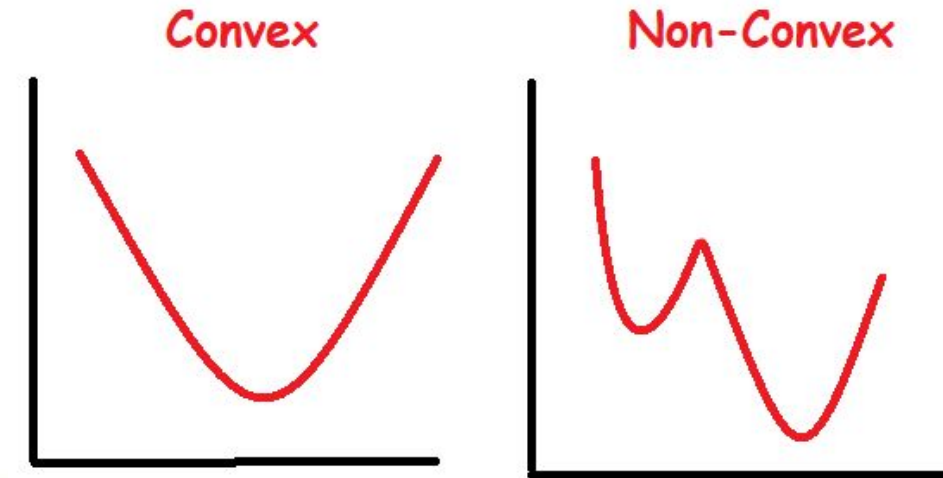




Convexity

How do you know if you've found the **global minimum**, and not just a **local minimum**?

A convex function has only one minimum:



MSE is a convex cost function

- Only 1 minimum
- We just have to find it!

Gradient Descent Convergence

Gradient Descent is guaranteed to eventually find the minimum, if:

1. A global minimum exists, with no local minima
 2. An appropriate value of α is chosen
- 

Stopping Criteria



- If the minimum is found, you're done ✓
- But, it's not uncommon for Gradient Descent to take a long time to converge (find a minimum where the derivative is exactly 0)
- Close enough is usually good enough
 - We can stop once we are sufficiently close to 0.
 - Simply calculate the Euclidean norm (size of the gradient vector) and check if less than a chosen threshold.
- Alternatively, you can:
 - Set a maximum number of iterations.
 - Stop when improvement drops below a chosen threshold

Gradient Ascent

There are times when instead of a minimum we want to find a **maximum**.

The same technique applies, but we just update the parameters in the opposite direction (**same direction as the gradient**).



Utility of Gradient Descent

Gradient Descent isn't the most powerful optimization technique, but it is perhaps the most **illustrative**.

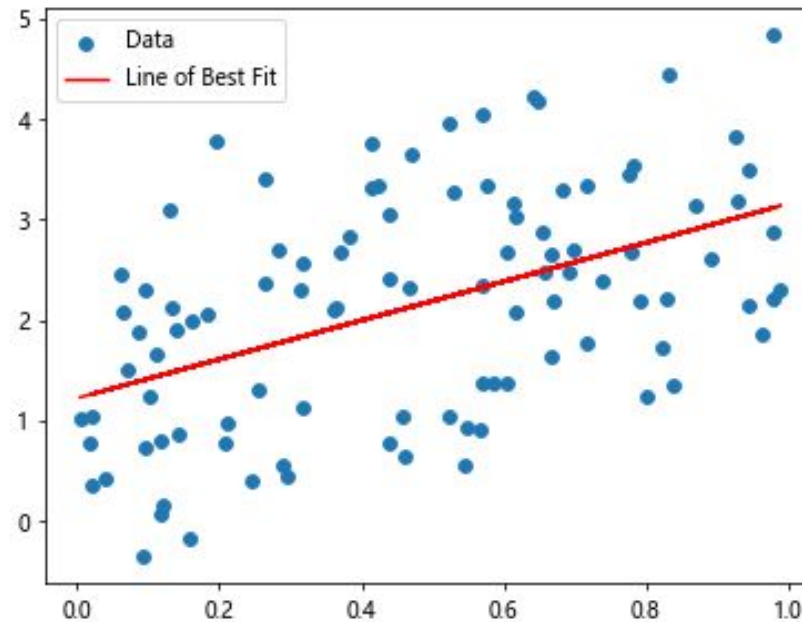
Understanding Gradient Descent well will give you strong intuition when doing optimization problems.



Linear Regression Output

Final State: Line of best fit

Once we're done, we should have a nice regression line (or hyperplane) we can use to make predictions.



Linear Regression Assumptions

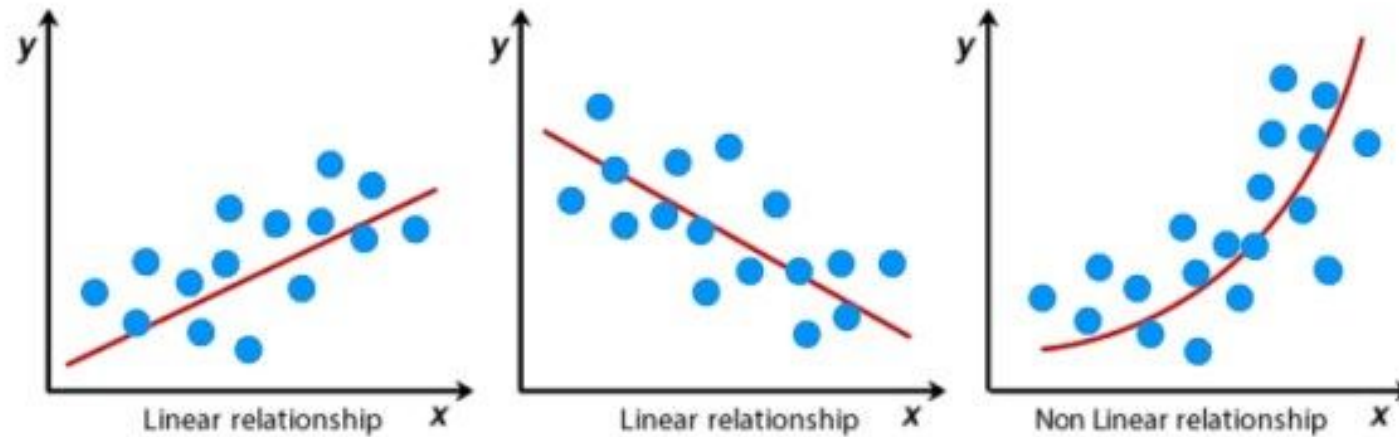


Linear Regression Assumptions (L.I.N.E.)

1. **Linearity**: Relationships between independent and dependent variables must be linear
 2. **Independence**: Observations are independent of each other, and residuals are independent of each other
 3. **Normality**: Residual errors are normally distributed
 4. **Equal variance** of residuals (homoscedasticity)
- 

Linearity

The relationship between the independent and dependent variables must be linear.



Independence

Observations must be independent of one another.

Residual Errors must be independent of one another.

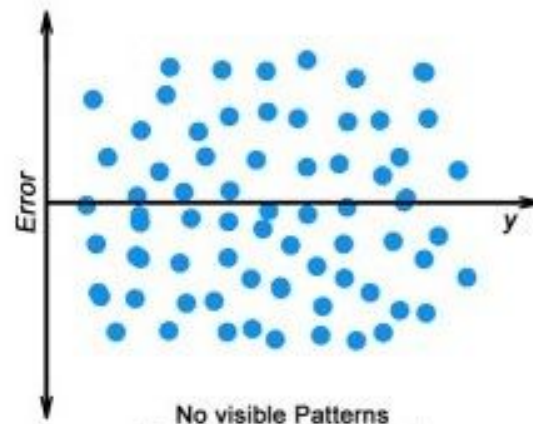
- There should be **no autocorrelation** in residuals for a linear regression model to be valid.



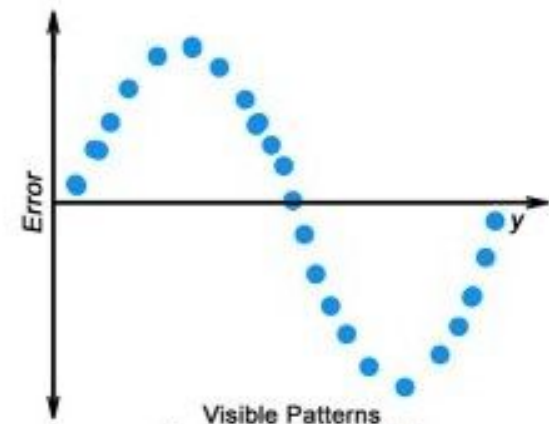
What is Autocorrelation?

Autocorrelation occurs when residuals are correlated with each other.

Ex: In predicting students' exam scores based on study hours, if errors in predicting one student's score are consistently related to errors in predicting another student's score, it implies autocorrelation. This violates the assumption of independent errors in linear regression.



No visible Patterns
Error Terms Independent



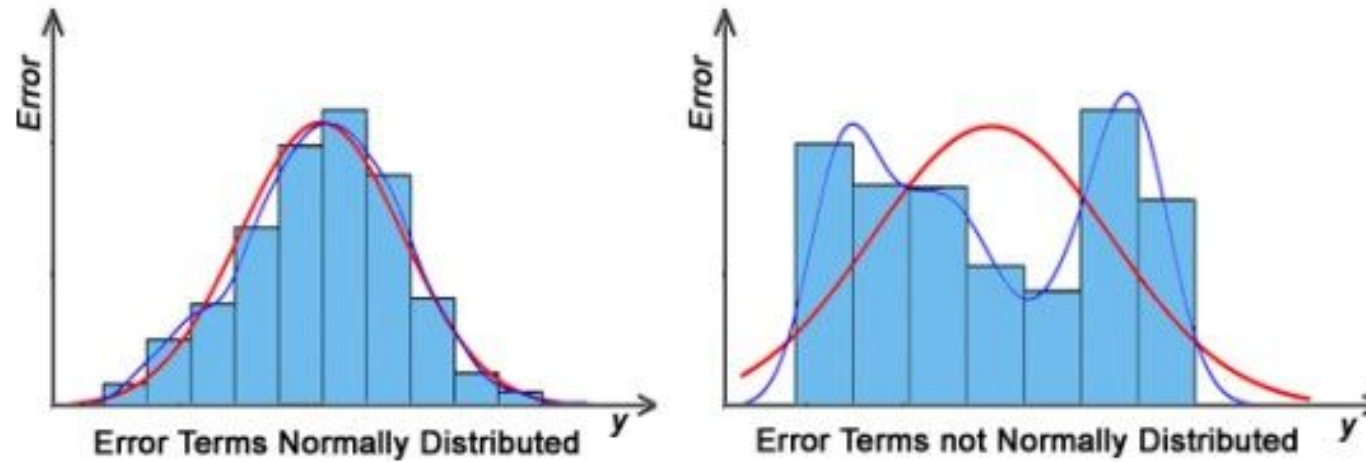
Visible Patterns
Error Terms dependent

- Autocorrelation usually occurs if there is a dependency between residual errors
- Problem: any correlation in the error term will drastically reduce the accuracy of the model.

Normality

Residuals must be normally distributed (or close to normally distributed).

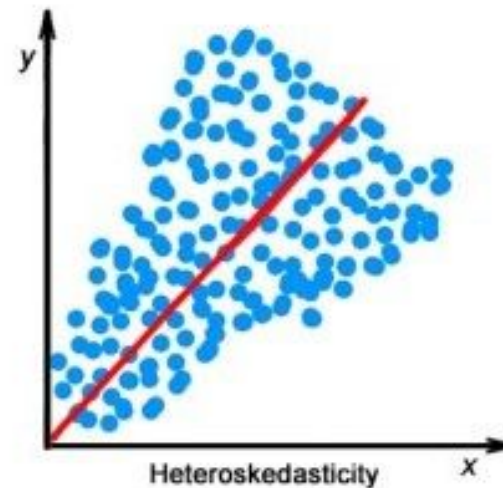
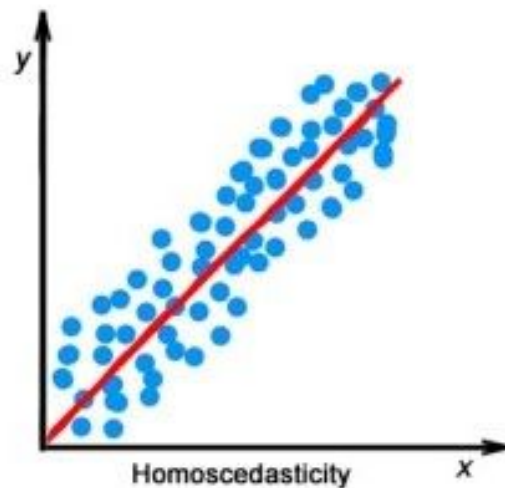
If they are not, you're likely to have outliers.



Equal variance of residuals (homoscedasticity)

The data must be homoscedastic (or close to homoscedastic), meaning the variance in the outliers must be more or less constant.

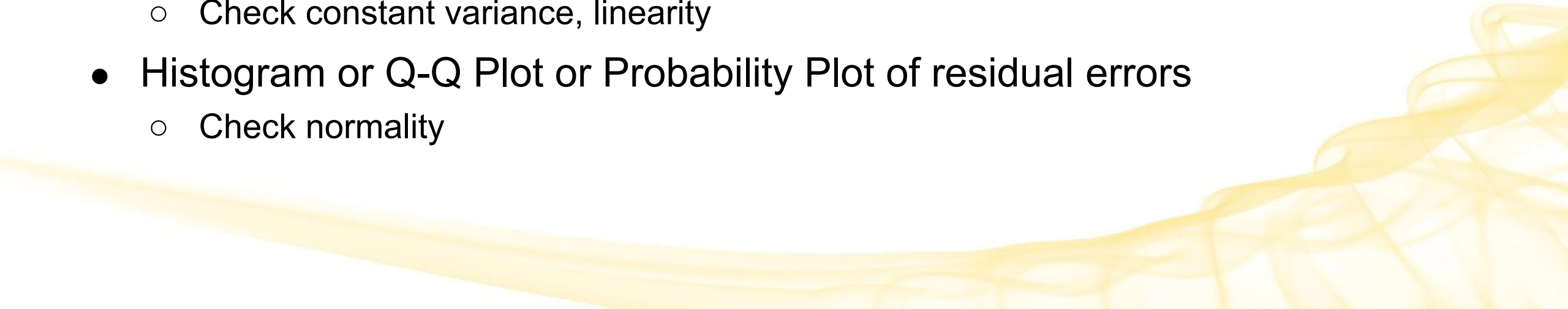
- Residuals should have constant variance across all levels of the independent variable.



Checking assumptions

Simple plots of the data and/or results can help us check our assumptions.

Examples:

- Scatter plot of x vs. y
 - Check linearity before fitting the regression model.
 - Plot of residual errors vs. predictor variable
 - Check constant variance, linearity
 - Histogram or Q-Q Plot or Probability Plot of residual errors
 - Check normality
- 

Evaluating Linear Regression



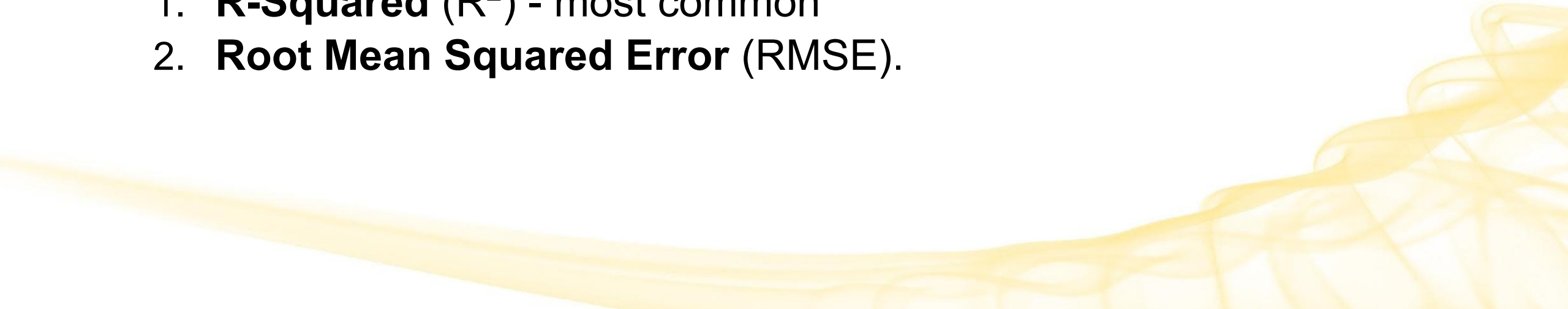
How do we know if our Regression is **any good**?



Evaluating Linear Regression Performance

After building a linear regression model, it is important to evaluate its performance.

The most commonly used metrics for evaluating the performance of linear regression models are:

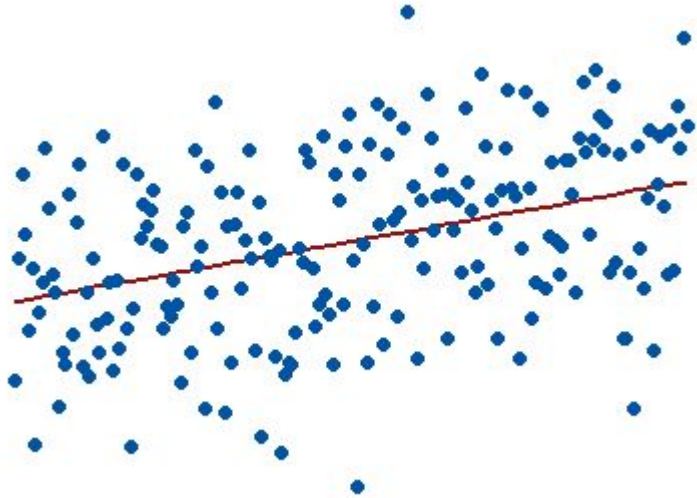
1. **R-Squared (R^2)** - most common
 2. **Root Mean Squared Error (RMSE)**.
- 

R²

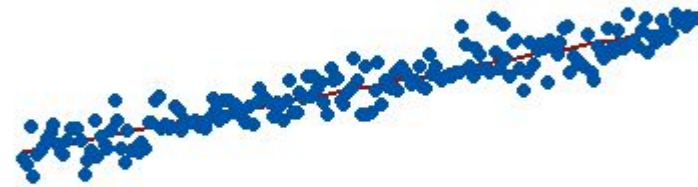
R-squared is a statistical method that **determines the goodness of fit.**

- Also called the **coefficient of determination**, or **coefficient of multiple determination** for multiple regression
- Tells us how well the regression line explains the variation between the actual values and our predicted output
 - Said another way, it **evaluates how scattered the data points are around the regression line**

R^2 Visualized



$R^2 = 15\%$



$R^2 = 85\%$

Image source: <https://statisticsbyjim.com/regression/interpret-r-squared-regression>

R^2

- Shows how well our line fits the data.
- **Range:** between 0 and 1
 - Typically reported as a **percentage** (ex. $R^2 = 0.92 = 92\%$)
 - Convenient because does not depend on the units of y
- Can be negative if the fit is worse than a horizontal line (*will examine*)

Interpretation:

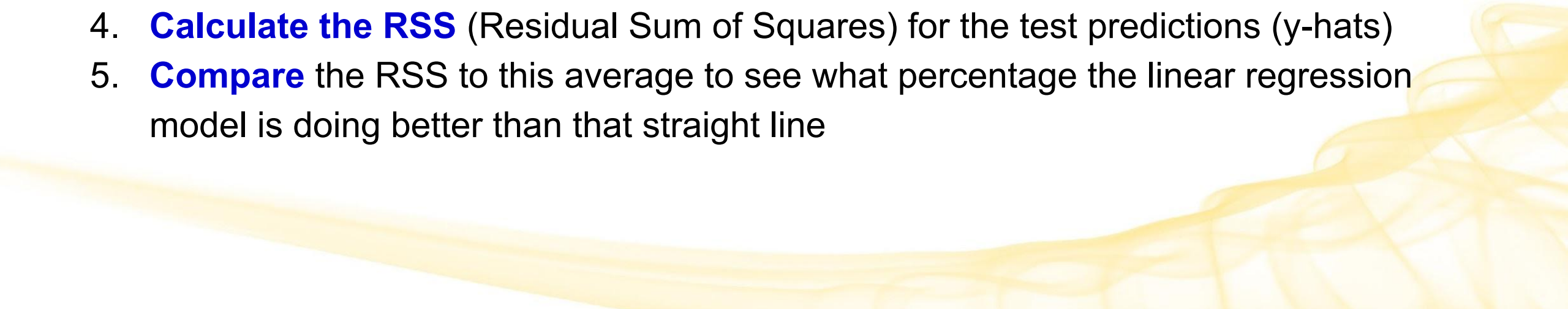
- **$R^2 = 0$:** Weak modeled relationship (the model doesn't explain any variation).
- **$R^2 = 1$:** Strong modeled relationship (the model perfectly explains all variation).
- **Higher R^2 :** Means the model does a better job of predicting the dependent variable
 - With caveats (*will discuss in a few*)

R^2

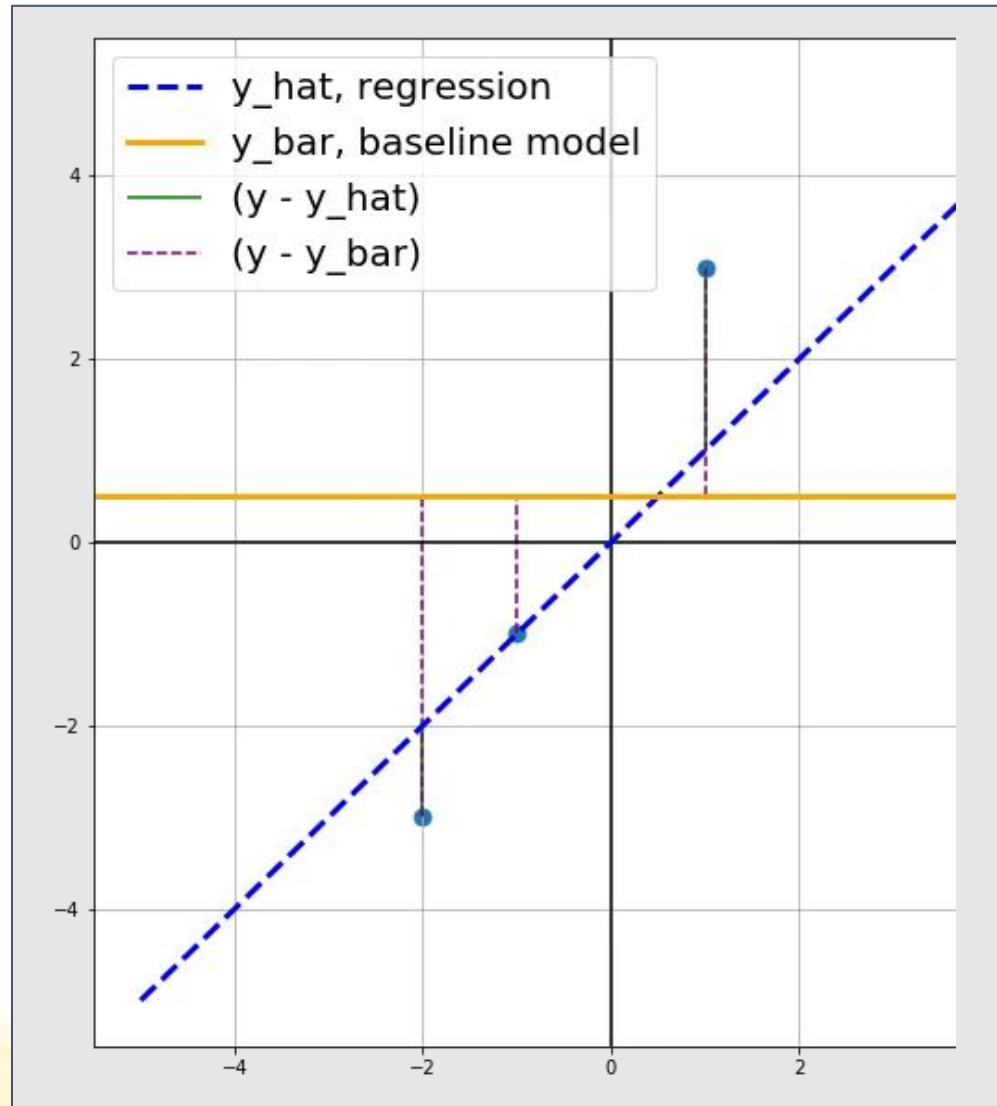
Another way to think about this:

R^2 tells us how good our regression line is at fitting (being **close, distance-wise** to) the data points, versus just using a horizontal line at the mean of the y-values.

Approach:

1. **Create a linear regression** model using training data
 2. **Make predictions** on the test data (create all the estimated y-hats) using the model
 3. **Create a horizontal line** reflecting the average (mean) of the actual test y-values
 4. **Calculate the RSS** (Residual Sum of Squares) for the test predictions (y-hats)
 5. **Compare** the RSS to this average to see what percentage the linear regression model is doing better than that straight line
- 

R² Calculation Visual



Source: <https://medium.com/@benroyalalexander/understanding-r-squared-visually-71f22d7d3279>

R² Calculation Visual

This distance is our residual error, as before.

$$\varepsilon_i = y_i - \hat{y}_i$$

It is error that is **not explained** by our model.



This distance is the distance between the regression line and the actual y mean.

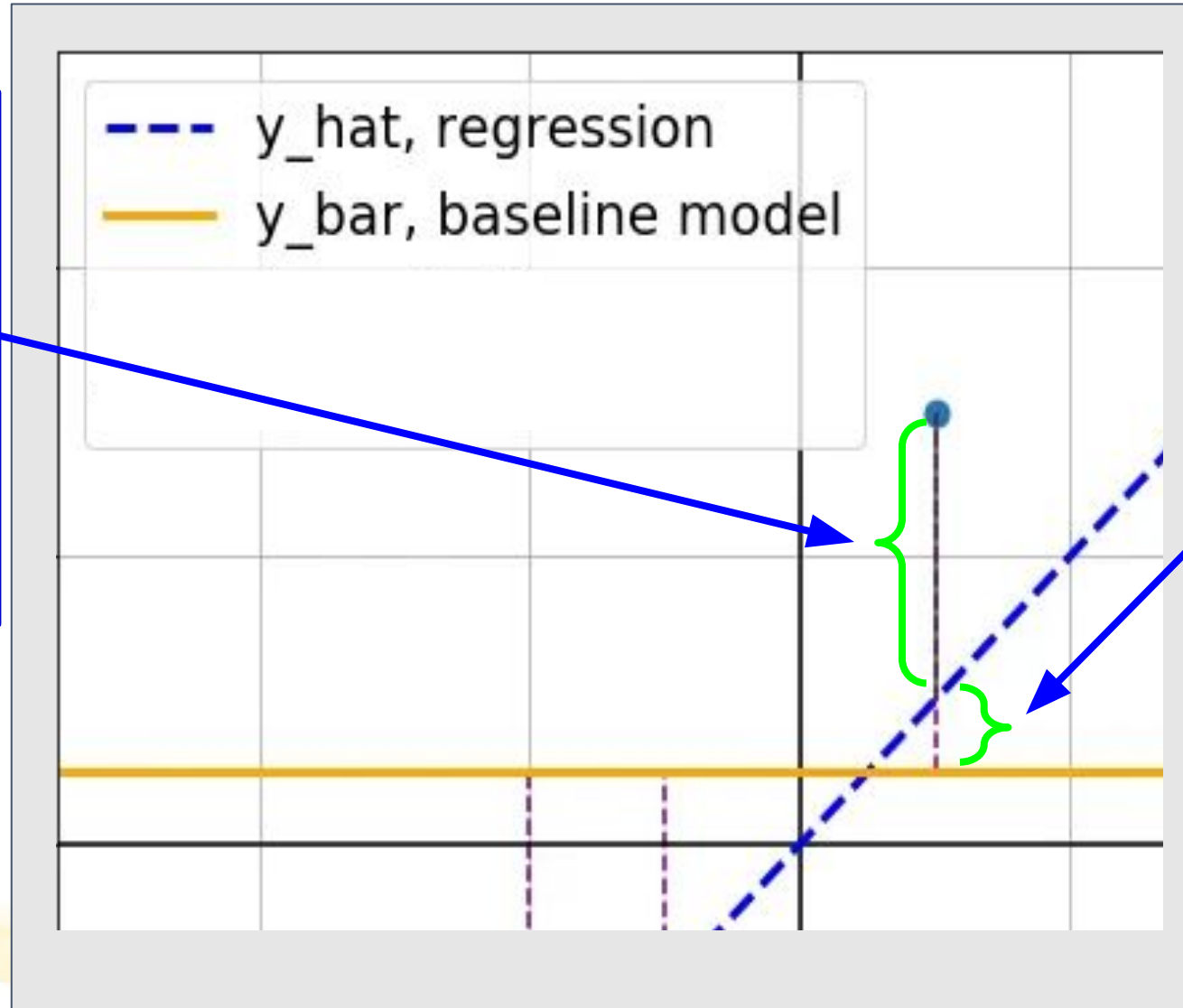
$$\hat{y}_i - \bar{y}_i$$

It is distance that **is explained** by our model.

R² Calculation Visual

Adding up the squares of all these residuals gives us our usual **RSS**

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Adding up the squares of these distances gives us ESS - the “Explained Sum of Squares”

$$ESS = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

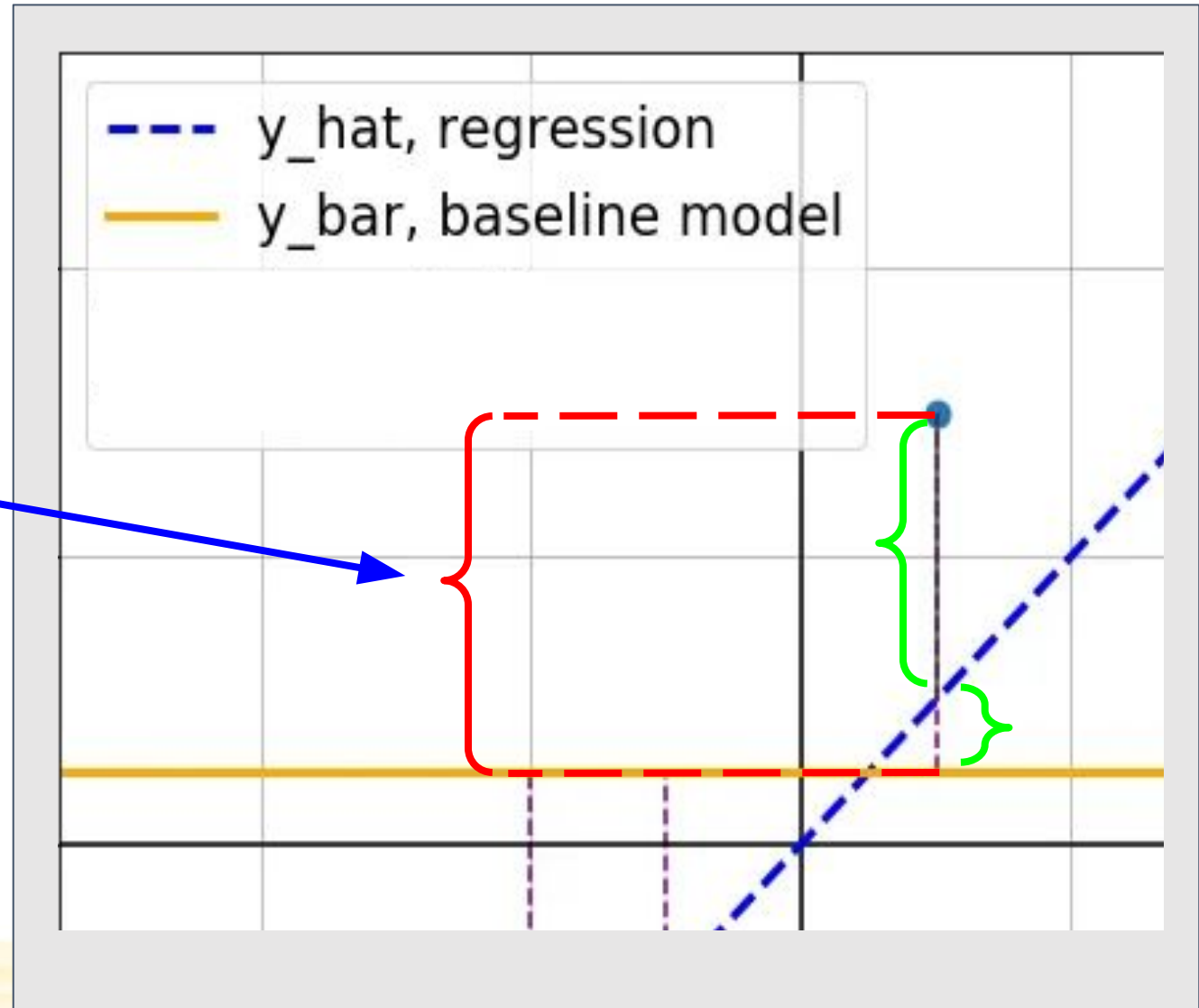
R² Calculation Visual

Adding these two together gives us the total difference between the actual y and the mean horizontal line.

$$y_i - \bar{y}_i$$

Adding up their sums of squares gives us the Total Sum of Squares (TSS)

$$TSS = RSS + ESS = \sum_{i=1}^n (y_i - \bar{y}_i)^2$$



R² Calculation

$$R^2 = 1 - \frac{\text{Unexplained variation}}{\text{Total Variation}} = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y}_i)^2}$$

This is equivalent to:

$$R^2 = \frac{\text{Explained variation}}{\text{Total Variation}} = \frac{ESS}{TSS} = \frac{\sum_{i=1}^n (\hat{y}_i - \bar{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y}_i)^2}$$

But what makes R^2 “squared”?

- **Recall:** Pearson Correlation Coefficient (r) tests for **linear correlation** between 2 variables
 - Tells us the strength and direction of a linear relationship between 2 variables (ex. x and y) → the degree to which 2 variables move together
 - Values are between -1 and 1
- **Under certain conditions, $R^2 = r^2$**
 - In simple linear regression
 - i.e. when fitting a straight line with a slope and intercept for 1 independent variable and 1 dependent variable (i.e. x and y)
 - We won't go over the proof here – feel free to explore
 - Maps r in $[-1, 1]$ to r^2 in $[0, 1]$

How can R^2 ever be negative?

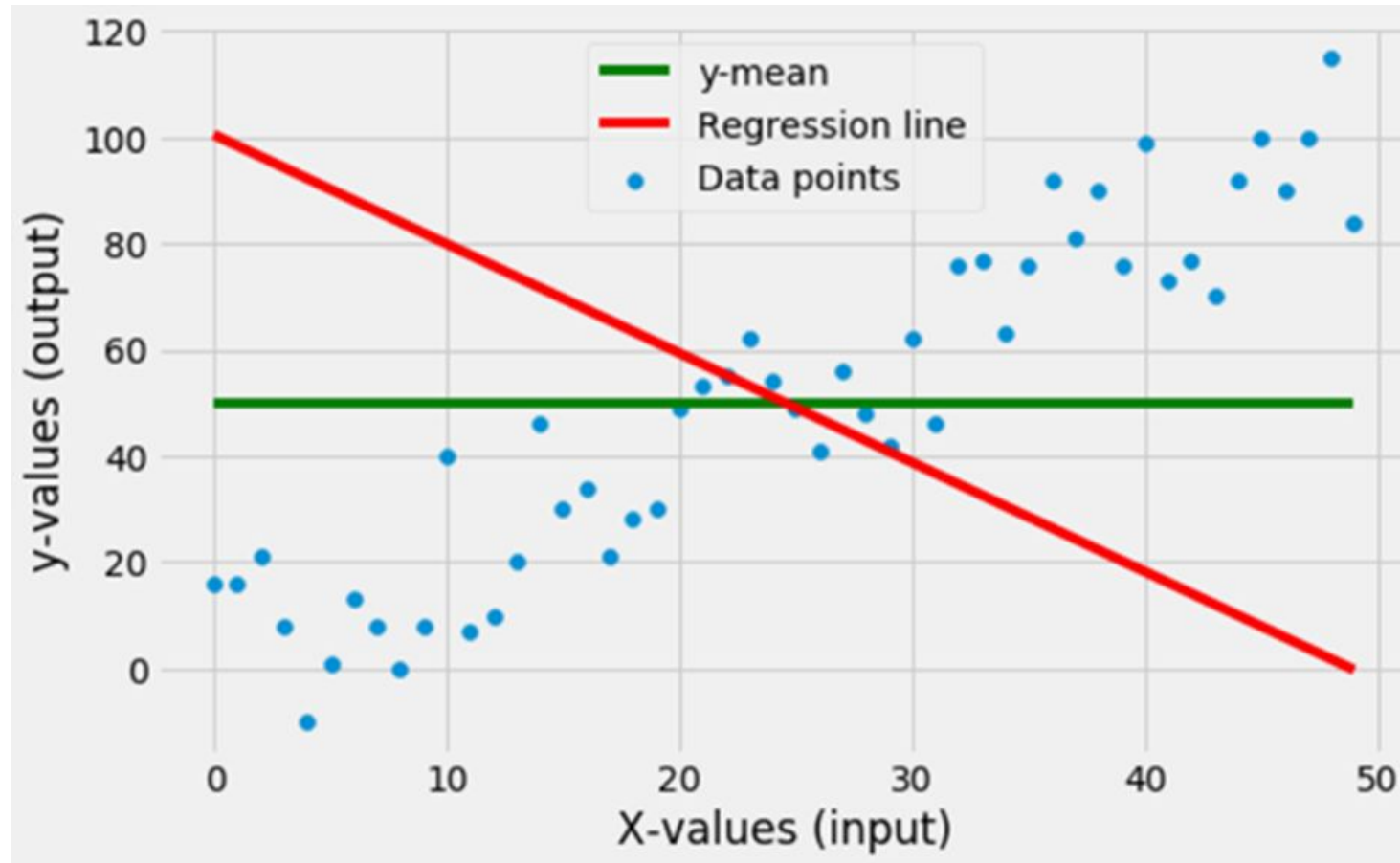
Recall: $R^2 = 1 - \frac{RSS}{TSS}$ $\longrightarrow$ If $RSS > TSS$, R^2 will be negative.

- But how can RSS be $> TSS$ if $TSS = RSS + ESS$?
- We previously assumed that our regression line would be **better** than the horizontal mean line.
- If instead, it's actually **worse**:
 - Residuals (distances between actual and predicted y-values) will be bigger than the distance between the actual y-values and the mean line.

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y}_i)^2}$$

$\longleftarrow$ This will be bigger
 $\longleftarrow$ than this

Negative R^2 Visualized



Rare, but can
and does
happen.

Image source: <https://medium.com/data-science-insights-and-predictions/a-negative-value-of-r-square-in-the-data-science-world-is-it-a-myth-fa6488e69efc>

R² Python Example

```
from sklearn.metrics import r2_score

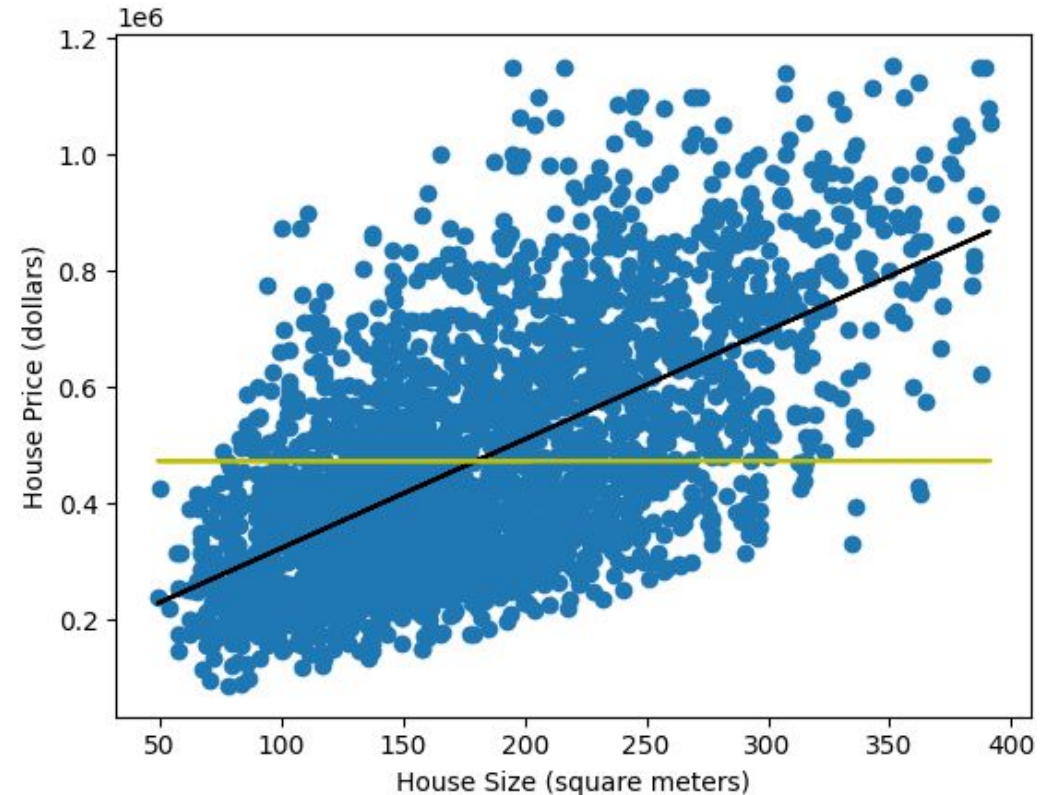
# Create a model, train it, and predict
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Compute R2
print('R2:', r2_score(y_test, y_pred))
```

0.379222179436684

```
# Create a mean line
mean = y_test.mean(axis=0)
mean_line = np.full(
    shape=len(y_test),
    fill_value=mean
)

# Plot regression line and mean line
plt.scatter(X_test, y_test)
plt.plot(X_test, y_pred, color='k')
plt.plot(X_test, mean_line, color='y')
plt.xlabel('House Size (square meters)')
plt.ylabel('House Price (dollars)')
plt.show()
```



Some Key Limitations of R^2

1. There is a limit to the meaning of R^2
 2. There is a limit to the meaning of r
 3. Correlation does not imply causation
- 

Limit to the meaning of R^2

A **low R^2 value** only tells us that the linear regression predictions do not do a good job of approximating the real data points – there is **too much spread** around the regression line.

- The line may still be the best possible linear fit of the data
- It could be that:
 - The relationship between the dependent and independent variables is indeed linear, but the data itself has a certain amount of randomness
 - The relationship is non-linear
 - You didn't have enough training data
 - Something else
- Low R^2 isn't necessarily always bad
 - Some areas inherently have a greater amount of unexplainable variation
 - Ex. human behavior is notoriously hard to model

Limit to the meaning of R^2

A high R^2 value isn't necessarily always good.

- Example possible issues:
 - Sample not representative of the population
 - Overfitting
 - Your model is fitting to noise/random errors, inflating the value of R^2
 - Poor choice of predictor variable
 - Output is actually dependent on a different form of the input variable
 - Ex. poverty rate as a predictor of income
- A reasonable value for R^2 may be understood from your domain knowledge
 - If you know your area inherently has a lot of unexplained variation, a high R^2 value could be a flag that something is off and you should investigate

Limit to the meaning of r

Both R^2 and r only test **linear** relationships between variables

- A **low r value** only tells us that the data suggests the variables are ***not linearly correlated***
 - This does not tell us that the variables are not correlated whatsoever
 - Nor that they're necessarily independent
 - 2 variables can have zero correlation but still be related if their relationship is non-linear
 - Example: consider data generated from the function $y = \sin(x)$
 - Significant in time series analysis due to seasonality
 - Clearly, x and y are perfectly correlated (in the sense that knowledge of x allows perfect prediction of y), but the linear correlation will be 0

Caution: Correlation vs Causation vs Regression

Correlation

- Degree and direction to which variables change together, indicating a relationship without implying a cause-and-effect.
- Ex. Ice cream sales and crime rates could be correlated because both tend to increase in summer, but one doesn't imply/cause the other

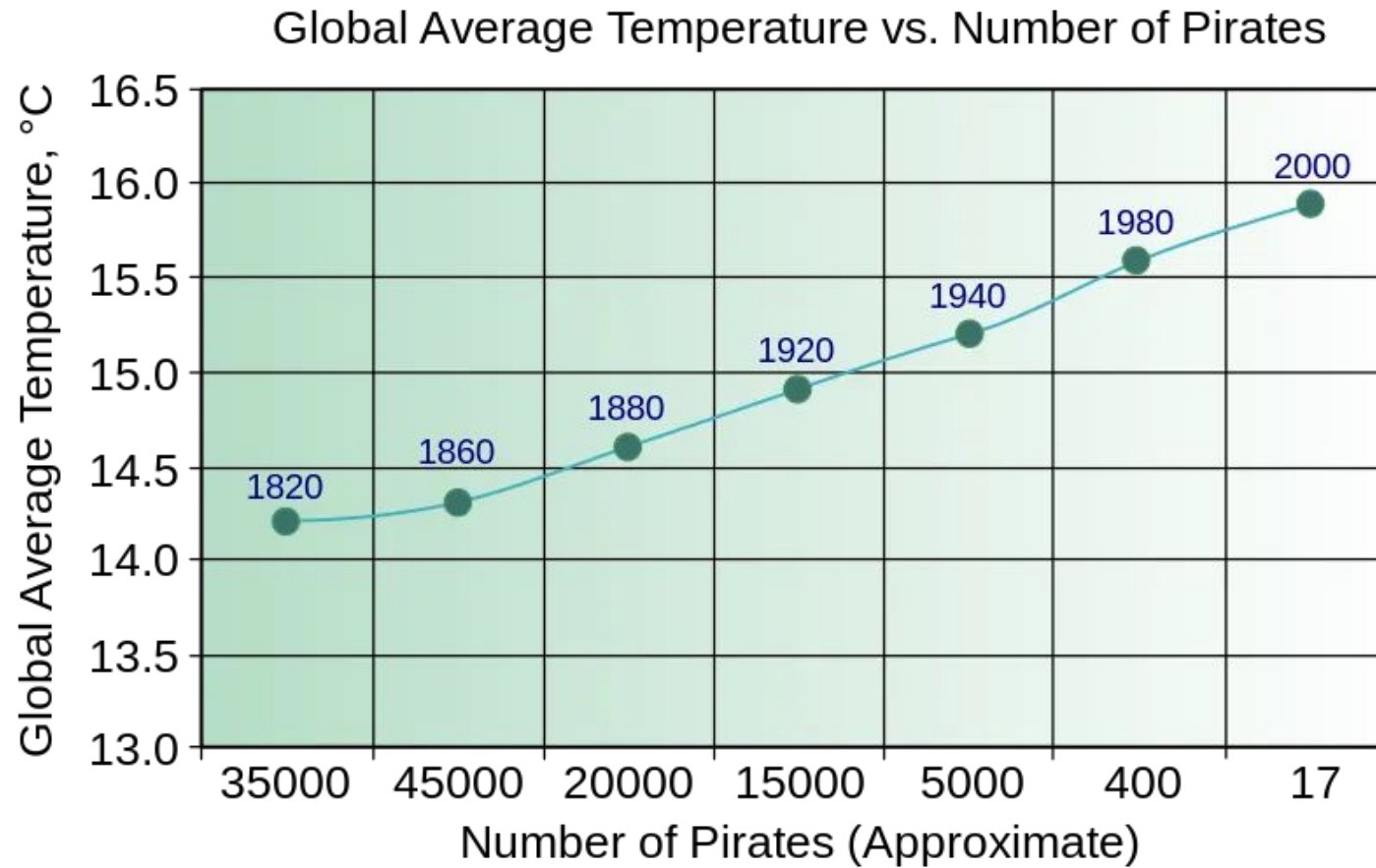
Causation

- Cause-and-effect relationship between variables, where changing one variable directly results in a change in the other.
- Ex. Increasing number of hours studying increases exam grades

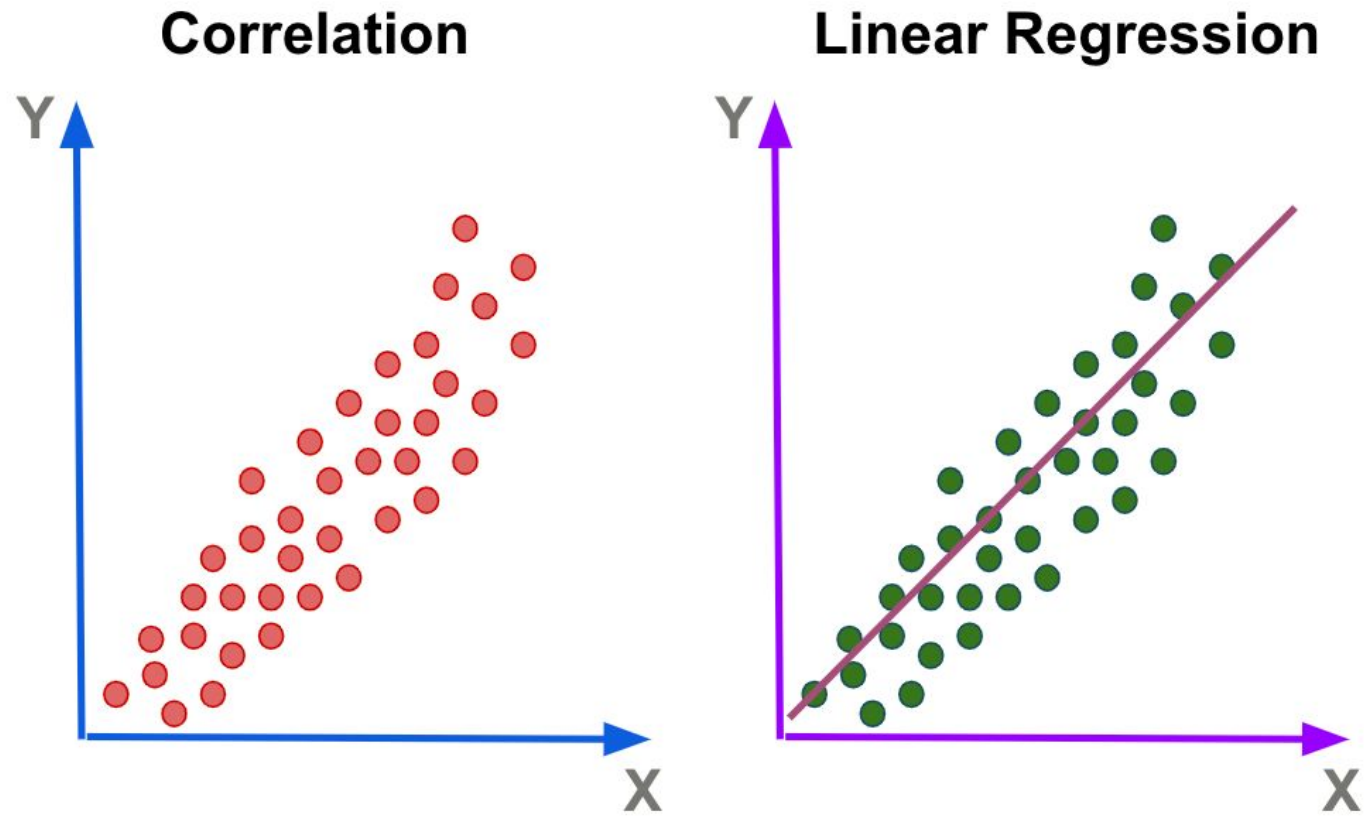
Regression

- Models the relationship between variables, enabling prediction of an output variable previously unseen observations of input variables.
- Can be used to explore potential causal relationships, but is more closely related to correlation.
- Ex. Predicting a person's height based on their weight

Canonical Example: Causation vs Correlation



Correlation vs Regression

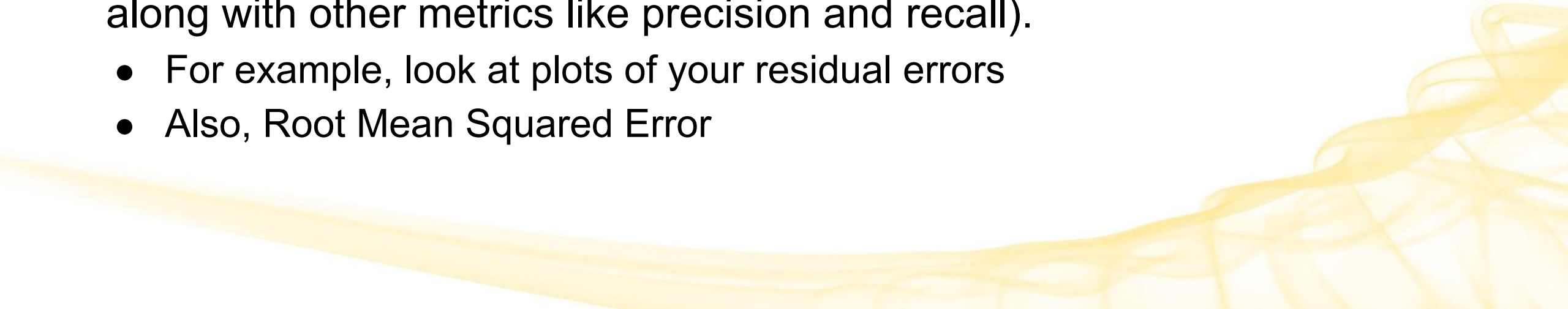


R² Takeaways

R² can be very useful, but you have to understand what you're doing and what the number is telling you (or not telling you).

- Interpretation is not always straightforward.

Best thing to do is **use it as one data point, combined with other things** to aid your understanding (similar to how we used accuracy along with other metrics like precision and recall).

- For example, look at plots of your residual errors
 - Also, Root Mean Squared Error
- 

Root Mean Squared Error

We used Mean Squared Error as our cost function for Linear Regression.

- We can use the square root of MSE, RMSE, as a measure of goodness of fit.
 - Measure of the average difference between predicted and actual values
 - Represents the standard deviation of the residuals (how dispersed the errors are)
 - Less error → lower RMSE. Closer to zero is better.
- We take square root just to get it into the same units as the data
- **Good at comparing two models for the same data**
- But generally useless on its own because it's non-standardized, vs R^2 which is standardized onto $[0, 1]$

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Linear Regression

Practical Tips and Gotchas

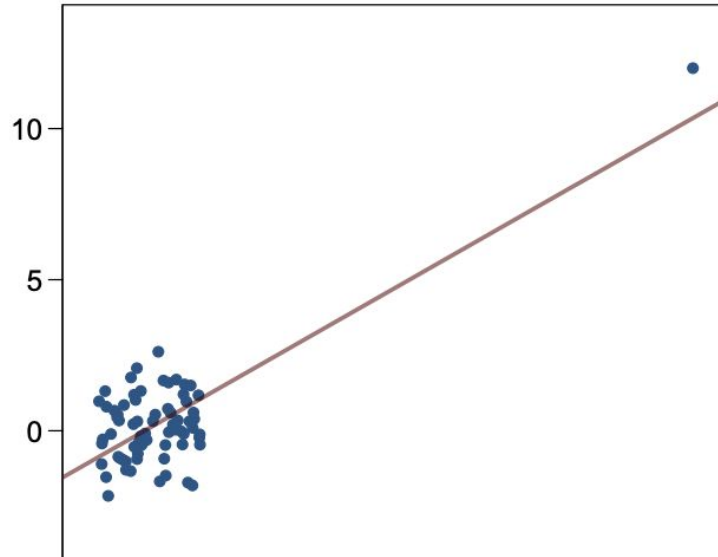


Careful: Linear Regression is Sensitive to Outliers

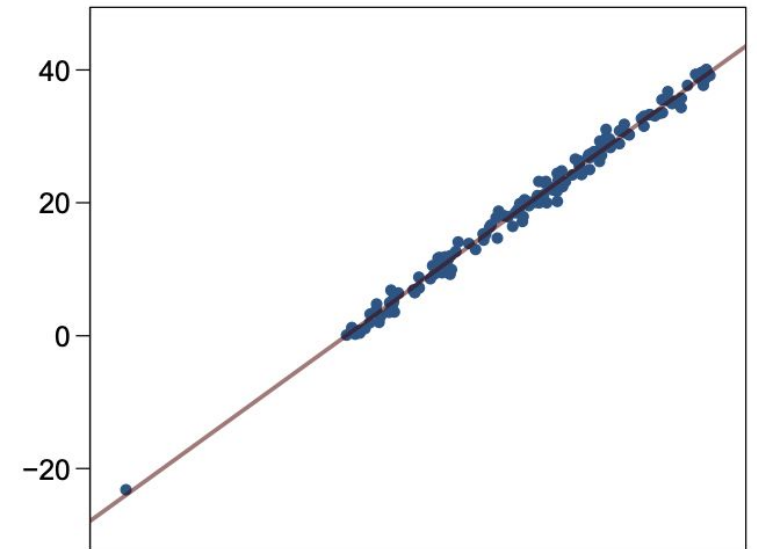
When outliers are present, linear regression tends to shift towards them.

Consider:

High-leverage data points that influence the **slope** of the regression line are called **influential points**.



Not all outliers are influential points



Linear Regression is Sensitive to Outliers

In practical terms, this is something to be aware of because it influences:

- Your model
- Your model evaluation (both R^2 and RMSE are affected by outliers)

However, we know from the Data Cleaning lecture that we can't just blindly remove outliers.

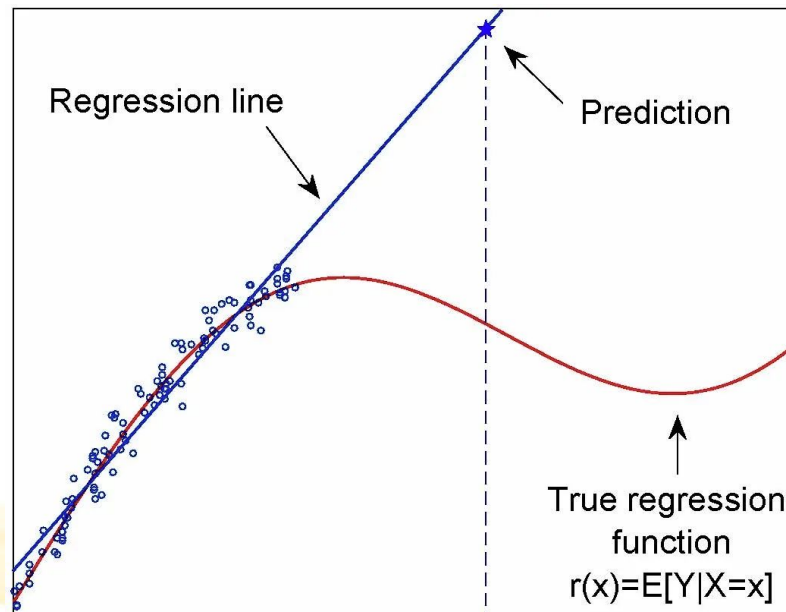
- Sometimes they can be removed, but not always
- Sometimes outliers represent something important in the data that we don't want to miss

We must be thoughtful in how we understand, interpret, and handle outliers.

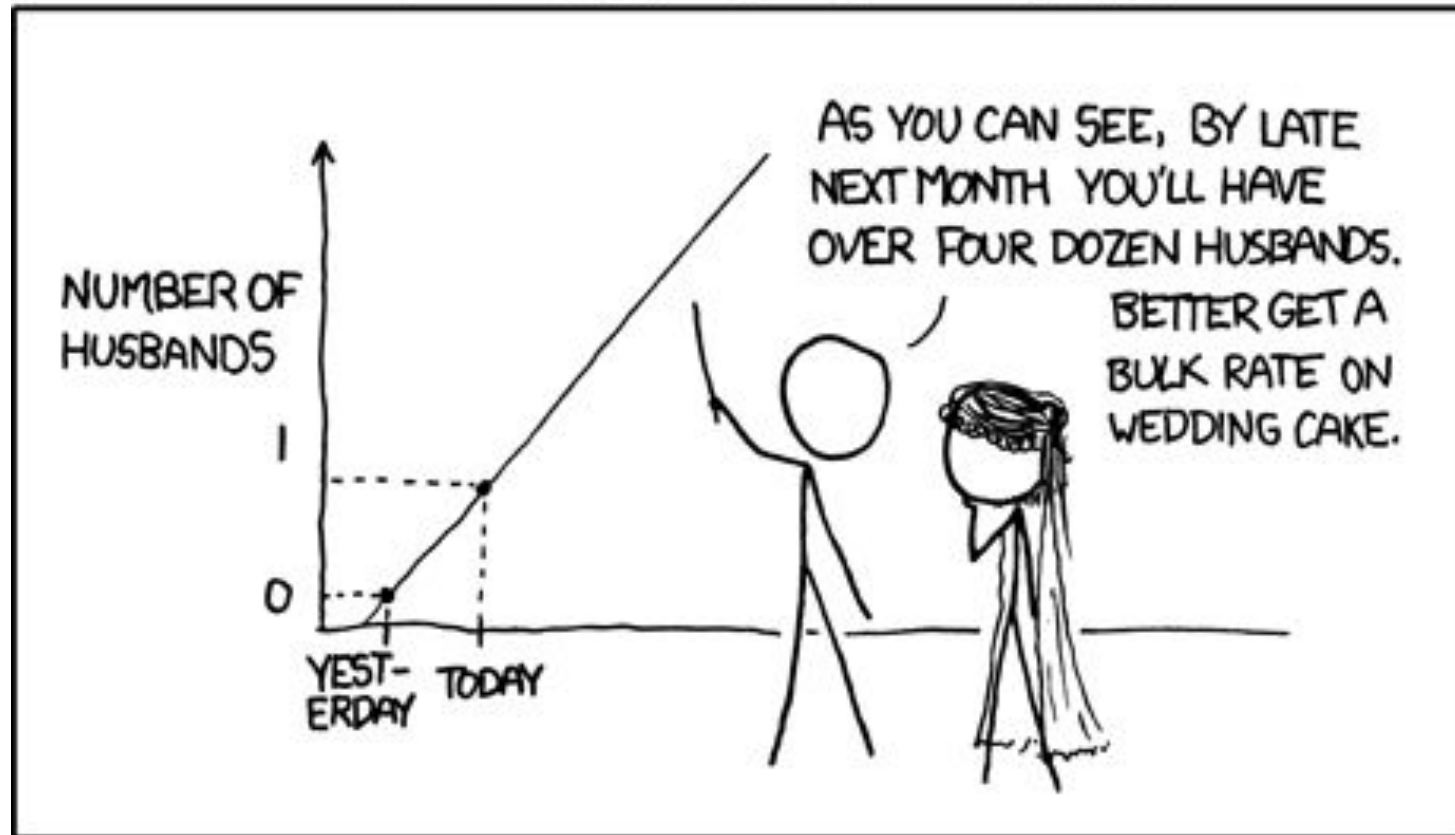
Careful: Extrapolation

Extrapolation is when we use the regression line to estimate values **well outside** of the range of the original data.

- **This is dangerous to do**
 - Results are generally inaccurate and cannot be trusted
 - Results can be misleading and mislead others
- Illustrative example:



Extrapolation



Tip: Normalization

Remember that with multiple linear regression we are fitting a line given by $\theta_0 + \theta_1 x_1 + \dots \theta_n x_n$

- What happens if the range of say, x_1 , is way bigger than the range of all the other x values?
 - x_1 would have a **disproportionate** effect on the MSE/cost calculation
- To get a good fit, it can be helpful to rescale so that they're all at the same comparable magnitude
 - We can do this using the same feature scaling techniques as before
 - Ex. Min-Max Normalization

Tip: Qualitative Features

So far, we've assumed all of the features we used are **quantitative**, or numerical.

However, using techniques we've already learned about, we can extend linear regression to include **qualitative** features, like categoricals.

- Example: One Hot Encoding to convert categorical variables into binary vectors
 - Ex. Encoding days of the week: Tuesday represented by $[0, 1, 0, 0, 0, 0, 0]$

Again the key is to understand what we are trying to do and choose an appropriate method.